TRƯỜNG ĐẠI HỌC NAM CẦN THƠ

KHOA CÔNG NGHỆ THÔNG TIN

NGUYỄN HỮU THẮNG

236690

LỚP: DH23TIN09

TÊN ĐỀ TÀI:
HỆ THỐNG TÓM TẮT VĂN BẢN

ĐỒ ÁN 1

Ngành: Công Nghệ Thông Tin

Mã số ngành: 7480201

CÁN BỘ HƯỚNG DẪN

ThS. TRẦN VĂN THIỆN

Tháng …/2026

TRƯỜNG ĐẠI HỌC NAM CẦN THƠ

KHOA CÔNG NGHỆ THÔNG TIN

NGUYỄN HỮU THẮNG

236690

LỚP: DH23TIN09

TÊN ĐỀ TÀI:
HỆ THỐNG TÓM TẮT VĂN BẢN

ĐỒ ÁN 1

Ngành: Công Nghệ Thông Tin

Mã số ngành: 7480201

CÁN BỘ HƯỚNG DẪN

ThS. TRẦN VĂN THIỆN

Tháng …/2026

PHIẾU CHẤM ĐIỂM

Sinh viên thực hiện:

Họ và Tên	Ghi chú	Chữ ký

Giảng viên chấm:

Họ và Tên	Ghi chú	Chữ ký
Giảng viên 1:		
Giảng viên 2:		

Cần Thơ,Thứ...., ngày .... tháng ... năm 2026

LỜI CẢM TẠ

Trong suốt quá trình thực hiện và hoàn thiện báo cáo Đồ án 1 với đề tài xây dựng "Hệ thống tóm tắt văn bản", em đã nhận được rất nhiều sự hỗ trợ, hướng dẫn và động viên từ quý thầy cô, gia đình và bạn bè.

Trước tiên, em xin chân thành cảm ơn quý thầy cô Khoa Công nghệ Thông tin – Trường Đại học Nam Cần Thơ đã tận tâm giảng dạy và trang bị cho em những kiến thức nền tảng cũng như chuyên môn trong suốt thời gian học tập. Đây chính là cơ sở quan trọng giúp em có thể tiếp cận, nghiên cứu và hoàn thành đề tài của mình.

Em xin gửi lời biết ơn sâu sắc đến ThS. Trần Văn Thiện – giảng viên hướng dẫn, người đã luôn theo sát, tận tình chỉ dẫn và đưa ra những góp ý thiết thực trong suốt quá trình thực hiện đồ án. Nhờ sự hỗ trợ của thầy, em đã có thể từng bước hoàn thiện đề tài từ việc nghiên cứu lý thuyết, phân tích yêu cầu đến xây dựng hệ thống và hoàn chỉnh báo cáo.

Đồng thời, em cũng xin cảm ơn gia đình và bạn bè đã luôn bên cạnh, động viên và tạo điều kiện thuận lợi để em có thể tập trung học tập và hoàn thành tốt đồ án này.

Mặc dù đã nỗ lực hết mình, nhưng do còn hạn chế về kiến thức và kinh nghiệm thực tiễn, báo cáo khó tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp quý báu từ quý thầy cô để đề tài được hoàn thiện hơn trong thời gian tới.

Em xin chân thành cảm ơn!

Cần Thơ, ngày..... tháng … năm 2026

Người thực hiện

LỜI CAM KẾT

Em xin cam kết đồ án "hệ thống tóm tắt văn bản" là kết quả của quá trình nghiên cứu, thiết kế và phát triển do chính em thực hiện, dựa trên kiến thức đã được học tại trường cùng với việc tự tìm hiểu và tham khảo các tài liệu, nguồn thông tin đáng tin cậy.

Toàn bộ nội dung trong đồ án như ý tưởng, mô hình, thuật toán, mã nguồn và dữ liệu đều do em tự xây dựng hoặc được tham khảo từ các nguồn có trích dẫn rõ ràng, tuân thủ đúng các quy định về bản quyền. Em cam kết không sao chép hay sử dụng trái phép bất kỳ tài liệu, sản phẩm hay công trình nghiên cứu nào của người khác.

Kết quả trình bày trong báo cáo cũng như sản phẩm phần mềm là trung thực, phản ánh đúng quá trình thực hiện đồ án, không có sự gian lận hay chỉnh sửa sai lệch vì bất kỳ mục đích nào. Em xin chịu hoàn toàn trách nhiệm trước quy định của Trường Đại học Nam Cần Thơ về nội dung đã thực hiện.

Trong trường hợp phát hiện có vi phạm liên quan đến tính trung thực hoặc bản quyền, em xin hoàn toàn chịu trách nhiệm và chấp nhận mọi hình thức xử lý theo quy định hiện hành.

Cần Thơ, ngày..... tháng … năm 2026

Người thực hiện

NHẬN XÉT CỦA GIẢNG VIÊN

.....
.....
.....

MỤC LỤC

LỜI CẢM TẠ i
LỜI CAM KẾT ii
NHẬN XÉT CỦA GIẢNG VIÊN iii
MỤC LỤC iv
DANH MỤC HÌNH ix
DANH MỤC BẢNG x
DANH MỤC TỪ VIẾT TẮT xi
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI 1
1.1. Đặt vấn đề 1
1.2. Lý do chọn đề tài 1
1.3. Mục tiêu đề tài 2
1.4. Đối tượng và phạm vi nghiên cứu 3
1.4.1. Đối tượng nghiên cứu 3
1.4.2. Phạm vi nghiên cứu 3
1.5. Phương pháp nghiên cứu 3
1.6. Ý nghĩa khoa học và thực tiễn của đề tài 3
1.6.1. Ý nghĩa khoa học 3
1.6.2. Ý nghĩa thực tiễn 4
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 5
2.1. Tổng quan về xử lý ngôn ngữ tự nhiên 5
2.2. Tổng quan về bài toán tóm tắt văn bản 5
2.3. Phân loại các phương pháp tóm tắt văn bản 6
2.3.1. Phương pháp tóm tắt trích xuất 6
2.3.2. Phương pháp tóm tắt sinh diễn giải 7
2.3.3. So sánh hai phương pháp 8
2.4. Quy trình xử lý văn bản trong hệ thống tóm tắt 8
2.4.1. Thu thập hoặc nhập dữ liệu đầu vào 8
2.4.2. Làm sạch dữ liệu 8
2.4.3. Chuẩn hóa văn bản 8
2.4.4. Tách câu 9
2.4.5. Tách từ và xử lý từ 9
2.4.6. Xác định nội dung quan trọng 9
2.4.7. Sinh bản tóm tắt 9

2.5. Một số kỹ thuật và thuật toán thường dùng 9
2.5.1. Tần suất từ 9
2.5.2. TF-IDF 10
2.5.3. TextRank 10
2.5.4. Chấm điểm câu 10
2.5.5. Độ tương đồng giữa các câu 10
2.5.6. Các mô hình học sâu 11
2.6. Công nghệ và công cụ sử dụng trong đề tài 11
2.6.1. Ngôn ngữ lập trình Python 11
2.6.2. Thư viện hỗ trợ xử lý văn bản 11
2.6.3. Công cụ quản lý phụ thuộc 11
2.6.4. Môi trường phát triển 11
2.6.5. Công cụ kiểm thử và chạy thử 11
2.7. Kết luận chương 12
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 13
3.1. Khảo sát bài toán và yêu cầu hệ thống 13
3.2. Phân tích yêu cầu chức năng 13
3.2.1. Chức năng nhập văn bản đầu vào 13
3.2.2. Chức năng kiểm tra dữ liệu đầu vào 14
3.2.3. Chức năng tiền xử lý văn bản 14
3.2.4. Chức năng tóm tắt văn bản 14
3.2.5. Chức năng hiển thị kết quả 14
3.2.6. Chức năng thao tác lại 14
3.3. Phân tích yêu cầu phi chức năng 14
3.3.1. Tính dễ sử dụng 14
3.3.2. Hiệu năng xử lý 14
3.3.3. Tính ổn định 15
3.3.4. Tính chính xác tương đối 15
3.3.5. Tính mở rộng 15
3.3.6. Tính bảo trì 15
3.4. Phân tích tác nhân và ca sử dụng 15
3.4.1. Tác nhân của hệ thống 15
3.4.2. Danh sách ca sử dụng 15
3.4.3. Biểu đồ Use Case 16
3.5. Thiết kế luồng xử lý hệ thống 17

3.6. Thiết kế kiến trúc hệ thống 17
3.6.1. Thành phần giao tiếp và điều phối 17
3.6.2. Thành phần xử lý tóm tắt 17
3.6.3. Thành phần quản lý phụ thuộc hệ thống 17
3.6.4. Thành phần hướng dẫn sử dụng 18
3.7. Thiết kế các module chức năng 18
3.7.1. Module giao diện người dùng 18
3.7.2. Module điều khiển hệ thống 18
3.7.3. Module xử lý tóm tắt văn bản 18
3.8. Thiết kế dữ liệu đầu vào và đầu ra 19
3.8.1. Dữ liệu đầu vào 19
3.8.2. Dữ liệu đầu ra 19
3.9. Kết luận chương 19
CHƯƠNG 4. XÂY DỰNG HỆ THỐNG 20
4.1. Môi trường và công cụ phát triển 20
4.2. Cấu trúc mã nguồn chương trình 20
4.3. Cài đặt module giao diện và điều phối 21
4.4. Cài đặt giao diện các chức năng chính 22
4.4.1. Trang chủ 22
4.4.2. Trang công cụ 23
4.4.3. Trang hướng dẫn sử dụng 24
4.4.4 Trang về ứng dụng 25
4.5. Cài đặt module xử lý tóm tắt văn bản 25
4.5.1. Đếm token 25
4.5.2. Tách câu 26
4.5.3. Chia văn bản thành chunk 26
4.5.4. Tóm tắt từng đoạn văn bản 26
4.6. Mô tả chi tiết luồng xử lý chương trình 26
4.7. Kết quả đầu ra và cấu trúc dữ liệu trả về 27
4.8. Một số đặc điểm kỹ thuật nổi bật của hệ thống 28
4.8.1. Hỗ trợ nhiều mô hình pretrained 28
4.8.2. Hỗ trợ xử lý văn bản dài bằng chunking 28
4.8.3. Tự động chọn CPU hoặc GPU 28
4.8.4. Tối ưu trải nghiệm trên Streamlit Cloud 28
4.8.5. Giao diện tiếng Việt, xử lý văn bản tiếng Anh 29

4.9. Hướng dẫn cài đặt và chạy hệ thống 29
4.10. Đánh giá việc cài đặt theo repo thực tế 29
4.11. Kết luận chương 29
CHƯƠNG 5. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ 31
5.1. Mục tiêu thực nghiệm 31
5.2. Dữ liệu thực nghiệm 31
5.2.1. Văn bản ngắn 31
5.2.2. Văn bản trung bình 31
5.2.3. Văn bản dài 32
5.2.4. Dữ liệu kiểm thử lỗi 32
5.3. Kịch bản kiểm thử 32
5.3.1. Kiểm thử khởi động hệ thống 32
5.3.2. Kiểm thử tải mô hình 32
5.3.3. Kiểm thử với văn bản ngắn 32
5.3.4. Kiểm thử với văn bản dài 33
5.3.5. Kiểm thử cấu hình tham số 33
5.3.6. Kiểm thử tính năng xóa văn bản 33
5.3.7. Kiểm thử trên CPU/GPU 33
5.4. Kết quả thực nghiệm 34
5.4.1. Kết quả kiểm thử chức năng giao diện 34
5.4.2. Kết quả tải mô hình 35
5.4.3. Kết quả đếm token và gợi ý mô hình 36
5.4.4. Kết quả với văn bản ngắn 36
5.4.5. Kết quả với văn bản dài 37
5.4.6. Kết quả hiển thị thông tin hỗ trợ đánh giá 37
5.5. Đánh giá hệ thống 37
5.5.1. Đánh giá định tính 37
5.5.2. Đánh giá định lượng 38
5.5.3. Đánh giá trải nghiệm sử dụng 39
5.6. Ưu điểm của hệ thống 39
5.6.1. Có giao diện web hoàn chỉnh 39
5.6.2. Hỗ trợ nhiều mô hình 39
5.6.3. Có cơ chế chunking cho văn bản dài 39
5.6.4. Có khả năng hiển thị thông tin hỗ trợ đánh giá 39
5.6.5. Hỗ trợ CPU và GPU 40

5.6.6. Tài liệu hướng dẫn khá đầy đủ 40
5.7. Hạn chế của hệ thống 40
5.7.1. Chưa hỗ trợ tốt cho tiếng Việt trong phần xử lý 40
5.7.2. Phụ thuộc vào mô hình pretrained 40
5.7.3. Tốn tài nguyên khi dùng mô hình lớn 40
5.7.4. Chưa có bộ đánh giá tự động chuẩn hóa 40
5.7.5. Chưa hỗ trợ nhiều loại đầu vào 40
5.7.6. Chưa có cơ chế lưu lịch sử hoặc xuất kết quả 41
5.8. So sánh với mục tiêu ban đầu 41
5.8.1. Các mục tiêu đã đạt được 41
5.8.2. Các mục tiêu chưa hoàn thiện hoàn toàn 41
5.9. Kết luận chương 41
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 42
6.1. Những kết quả đạt được 42
6.2. Hạn chế còn tồn tại 42
6.3. Hướng phát triển 43
6.4. Kết luận 44
TÀI LIỆU THAM KHẢO 45

DANH MỤC HÌNH

Hình 3.1. Biểu đồ Use Case của hệ thống tóm tắt văn bản 16
Hình 4.1 menu điều hướng 21
Hình 4.2 giao diện trang chủ 22
hình 4.3 giao diện trang công cụ 23
Hình 4.4 giao diện trang hướng dẫn sử dụng 24
Hình 4.5 giao diện trang về ứng dụng 25
Hình 5.1 kết quả kiểm thử giao diện trang chủ 34
Hình 5.2 kết quả kiểm thử tải mô hình t5-small 35
Hình 5.3 kết quả kiểm thử chức năng đếm token 36
Hình 5.4 kết quả kiểm thử việc tóm tắt văn bản ngắn 36
Hình 5.5 kết quả kiểm thử việc tóm tắt văn bản dài 37

DANH MỤC BẢNG

Bảng 5.1. Thành phần dùng để tính accuracy trong benchmark nội bộ. 38
Bảng 5.2. So sánh độ chính xác và các thành phần đánh giá của hai mô hình. 38

DANH MỤC TỪ VIẾT TẮT

Thuật ngữ	Viết tắt	Ngữ nghĩa
Natural Language Processing	NLP	Bộ xử lý Ngôn ngữ Tự nhiên
Natural Language Toolkit	NLTK	Thư viện xử lý ngôn ngữ tự nhiên
Inverse Document Frequency	IDF	Độ hiếm của từ trong tập văn bản
Term Frequency	TF	Tần xuất từ xuất hiện
Central Processing Unit	CPU	Bộ xử lý trung tâm
Graphics Processing Unit	GPU	Bộ xử lý đồ họa
Repositories	Repo	Kho lưu trữ mã nguồn (github)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1. Đặt vấn đề

Trong thời đại công nghệ số hiện nay, lượng thông tin được tạo ra và lưu trữ dưới dạng văn bản ngày càng lớn. Các nguồn thông tin như báo điện tử, tài liệu học tập, văn bản hành chính, bài nghiên cứu, nội dung trên các trang web và mạng xã hội liên tục được cập nhật với số lượng rất lớn mỗi ngày. Điều này mang lại nhiều thuận lợi trong việc tiếp cận tri thức, nhưng đồng thời cũng đặt ra thách thức không nhỏ cho người dùng trong việc đọc, chọn lọc và nắm bắt nhanh những nội dung quan trọng.

Trên thực tế, không phải lúc nào người dùng cũng có đủ thời gian để đọc toàn bộ một văn bản dài. Trong nhiều trường hợp, nhu cầu chủ yếu của người đọc là nắm được các ý chính, nội dung cốt lõi hoặc thông tin quan trọng nhất của tài liệu. Vì vậy, việc rút gọn nội dung văn bản thành một bản tóm tắt ngắn gọn, súc tích nhưng vẫn giữ được thông tin cần thiết là một nhu cầu rất thực tế và có giá trị ứng dụng cao.

Từ đó, bài toán "tóm tắt văn bản" đã trở thành một hướng nghiên cứu và ứng dụng quan trọng trong lĩnh vực xử lý ngôn ngữ tự nhiên. Tóm tắt văn bản giúp hỗ trợ người dùng tiết kiệm thời gian đọc, nâng cao hiệu quả tiếp nhận thông tin, đồng thời góp phần cải thiện khả năng khai thác tri thức từ dữ liệu văn bản lớn. Hiện nay, các hệ thống tóm tắt văn bản được ứng dụng trong nhiều lĩnh vực như giáo dục, báo chí, truyền thông, tìm kiếm thông tin, hỗ trợ đọc tài liệu và phân tích dữ liệu.

Xuất phát từ nhu cầu thực tiễn đó, đề tài về xây dựng "hệ thống tóm tắt văn bản" được lựa chọn nhằm xây dựng một hệ thống có khả năng tiếp nhận văn bản đầu vào, thực hiện xử lý và tạo ra nội dung tóm tắt một cách tự động. Đề tài không chỉ mang ý nghĩa ứng dụng mà còn giúp vận dụng các kiến thức đã học trong quá trình đào tạo như lập trình, phân tích thiết kế hệ thống, xử lý dữ liệu và xây dựng phần mềm.

1.2. Lý do chọn đề tài

Việc lựa chọn đề tài xây dựng hệ thống tóm tắt văn bản tự động xuất phát từ nhiều lý do cả về mặt học thuật lẫn thực tiễn.

Thứ nhất, trong bối cảnh lượng thông tin văn bản tăng nhanh, người dùng ngày càng có nhu cầu tiếp cận thông tin một cách nhanh chóng và hiệu quả hơn. Việc đọc toàn bộ các văn bản dài gây mất nhiều thời gian, đặc biệt đối với sinh viên, giảng viên, nhân viên văn phòng hoặc những người thường xuyên phải xử lý tài liệu. Một hệ thống tóm tắt văn bản có thể hỗ trợ người dùng nắm bắt nội dung

chính của tài liệu trong thời gian ngắn, từ đó nâng cao hiệu quả học tập và làm việc.

Thứ hai, tóm tắt văn bản là một bài toán tiêu biểu trong lĩnh vực xử lý ngôn ngữ tự nhiên. Đây là một lĩnh vực đang phát triển mạnh và có vai trò quan trọng trong nhiều ứng dụng hiện đại. Thông qua việc thực hiện đề tài này, sinh viên có cơ hội tiếp cận và áp dụng các kiến thức về xử lý văn bản, thuật toán lựa chọn thông tin, tổ chức dữ liệu và xây dựng ứng dụng phần mềm.

Thứ ba, đề tài có tính ứng dụng thực tế cao và phù hợp với định hướng đào tạo ngành Công nghệ thông tin. Không chỉ dừng ở mức nghiên cứu lý thuyết, đề tài còn cho phép triển khai thành một hệ thống cụ thể, có đầu vào, đầu ra rõ ràng và có thể kiểm thử, đánh giá được. Điều này giúp thể hiện được khả năng phân tích yêu cầu, thiết kế hệ thống, cài đặt chương trình và kiểm chứng kết quả.

Thứ tư, đề tài phù hợp với khả năng thực hiện trong khuôn khổ đồ án học phần. Hệ thống có thể được xây dựng bằng các công cụ và ngôn ngữ lập trình phổ biến như Python cùng với các thư viện hỗ trợ xử lý văn bản. Điều này tạo điều kiện thuận lợi cho việc triển khai, thử nghiệm và hoàn thiện sản phẩm trong thời gian cho phép.

Với những lý do trên, đề tài này là một lựa chọn phù hợp, vừa có giá trị thực tiễn, vừa có ý nghĩa học thuật, đồng thời đáp ứng tốt yêu cầu của một đồ án Công nghệ thông tin.

1.3. Mục tiêu đề tài

Đề tài được thực hiện với mục tiêu tổng quát là xây dựng một hệ thống có khả năng tóm tắt văn bản tự động, giúp người dùng rút gọn nội dung của một văn bản đầu vào mà vẫn giữ lại được các ý chính quan trọng.

Để đạt được mục tiêu tổng quát đó, đề tài tập trung vào bốn nhóm nhiệm vụ chính: tìm hiểu cơ sở lý thuyết về tóm tắt văn bản và các bước tiền xử lý cơ bản; phân tích yêu cầu và thiết kế một hệ thống phù hợp với phạm vi đồ án; xây dựng chương trình cho phép nhập văn bản đầu vào và sinh bản tóm tắt đầu ra; cuối cùng là kiểm thử trên một số văn bản mẫu để rút ra nhận xét về ưu điểm, hạn chế và hướng phát triển tiếp theo.

Thông qua việc hoàn thành các mục tiêu trên, đề tài không chỉ hướng tới việc xây dựng một sản phẩm phần mềm cụ thể mà còn giúp củng cố và vận dụng kiến thức chuyên ngành trong thực tế.

1.4. Đối tượng và phạm vi nghiên cứu

1.4.1. Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là bài toán tóm tắt văn bản tự động và quy trình xây dựng hệ thống phần mềm phục vụ bài toán này. Trong phạm vi thực hiện, báo cáo tập trung vào các nội dung cốt lõi gồm khái niệm và đặc điểm của tóm tắt văn bản, các bước tiền xử lý và chuẩn hóa dữ liệu, cách xác định nội dung quan trọng, quy trình thiết kế hệ thống và các công nghệ, thư viện phù hợp để cài đặt ứng dụng. Phần đánh giá hiệu quả được dựa trên các ví dụ thực nghiệm và kết quả đầu ra cụ thể của hệ thống.

1.4.2. Phạm vi nghiên cứu

Do giới hạn về thời gian, nguồn lực và khuôn khổ của một đồ án học phần, đề tài chỉ tập trung xây dựng hệ thống tóm tắt cho từng văn bản đầu vào tại một thời điểm, xử lý dữ liệu ở mức cơ bản bằng Python và các thư viện có sẵn, đồng thời minh họa rõ luồng xử lý từ đầu vào đến đầu ra để phục vụ mục đích học tập. Các hướng mở rộng như xử lý dữ liệu quy mô lớn, tóm tắt đa tài liệu, tích hợp mô hình học sâu yêu cầu tài nguyên cao hoặc triển khai trên hạ tầng thương mại chưa phải trọng tâm của phiên bản này.

1.5. Phương pháp nghiên cứu

Để thực hiện đề tài một cách có hệ thống, báo cáo kết hợp bốn hướng nghiên cứu chính. Thứ nhất là nghiên cứu lý thuyết để tổng hợp kiến thức nền về xử lý ngôn ngữ tự nhiên và tóm tắt văn bản. Thứ hai là phân tích và thiết kế hệ thống dựa trên yêu cầu chức năng, phi chức năng và mối liên hệ giữa các module. Thứ ba là thực nghiệm trực tiếp trên chương trình sau khi cài đặt để quan sát đầu ra, thời gian xử lý và khả năng đáp ứng của hệ thống. Cuối cùng là so sánh, đối chiếu kết quả với mục tiêu đề tài để rút ra nhận xét và đề xuất cải tiến.

1.6. Ý nghĩa khoa học và thực tiễn của đề tài

1.6.1. Ý nghĩa khoa học

Về mặt khoa học, đề tài góp phần làm rõ hơn quy trình xây dựng một hệ thống ứng dụng trong lĩnh vực xử lý ngôn ngữ tự nhiên, cụ thể là bài toán tóm tắt văn bản. Thông qua việc nghiên cứu lý thuyết và triển khai hệ thống thực tế, đề tài giúp hệ thống hóa các kiến thức liên quan đến xử lý văn bản, lựa chọn thông tin quan trọng, tổ chức luồng xử lý và xây dựng phần mềm.

Bên cạnh đó, đề tài còn là cơ hội để vận dụng và củng cố các kiến thức chuyên môn như lập trình Python, phân tích thiết kế hệ thống, tổ chức chương trình và kiểm thử phần mềm. Đây là nền tảng quan trọng để tiếp tục nghiên cứu sâu hơn

về các ứng dụng trí tuệ nhân tạo và xử lý ngôn ngữ tự nhiên trong tương lai.

1.6.2. Ý nghĩa thực tiễn

Về mặt thực tiễn, hệ thống tóm tắt văn bản có thể hỗ trợ người dùng tiếp cận thông tin một cách nhanh chóng hơn bằng cách rút gọn nội dung văn bản mà vẫn giữ lại các ý quan trọng. Điều này đặc biệt hữu ích trong bối cảnh người dùng thường xuyên phải tiếp xúc với số lượng lớn tài liệu như bài báo, tài liệu học tập, báo cáo hoặc nội dung trên Internet.

Ngoài ra, đề tài còn mang ý nghĩa ứng dụng trong học tập và làm việc, giúp giảm thời gian đọc, tăng hiệu quả nắm bắt thông tin và hỗ trợ xử lý dữ liệu văn bản trong nhiều tình huống thực tế. Mặc dù hệ thống trong phạm vi đồ án còn ở mức cơ bản, nhưng đây là tiền đề để phát triển thành các ứng dụng có khả năng xử lý thông minh hơn trong tương lai.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về xử lý ngôn ngữ tự nhiên

Xử lý ngôn ngữ tự nhiên NLP là một lĩnh vực thuộc trí tuệ nhân tạo và khoa học máy tính, tập trung vào việc xây dựng các phương pháp và hệ thống cho phép máy tính hiểu, phân tích, xử lý và sinh ra ngôn ngữ của con người. Ngôn ngữ tự nhiên ở đây bao gồm ngôn ngữ nói và ngôn ngữ viết được con người sử dụng trong đời sống hằng ngày như tiếng Việt, tiếng Anh, tiếng Pháp, tiếng Trung và nhiều ngôn ngữ khác.

Khác với dữ liệu có cấu trúc như số liệu hay bảng biểu, ngôn ngữ tự nhiên mang nhiều đặc điểm phức tạp như đa nghĩa, phụ thuộc ngữ cảnh, biến đổi linh hoạt về cú pháp và ngữ nghĩa. Vì vậy, để máy tính có thể xử lý tốt dữ liệu ngôn ngữ, cần phải áp dụng nhiều kỹ thuật khác nhau nhằm biến đổi văn bản thô thành dạng dữ liệu có thể phân tích được.

Hiện nay, xử lý ngôn ngữ tự nhiên đóng vai trò rất quan trọng trong nhiều hệ thống thông minh hiện đại. Các ứng dụng tiêu biểu có thể kể đến gồm dịch máy tự động, chatbot và trợ lý ảo, phân loại văn bản, phân tích cảm xúc, trích xuất thông tin, nhận dạng thực thể có tên, tìm kiếm thông minh và tóm tắt văn bản tự động. Sự đa dạng này cho thấy NLP không chỉ mang ý nghĩa học thuật mà còn có giá trị ứng dụng thực tế rất cao.

Trong các ứng dụng nêu trên, bài toán tóm tắt văn bản là một hướng nghiên cứu có tính ứng dụng cao, bởi nó giúp người dùng nhanh chóng nắm bắt nội dung quan trọng của tài liệu mà không cần đọc toàn bộ văn bản. Để xây dựng được một hệ thống tóm tắt văn bản, cần có nền tảng kiến thức về xử lý ngôn ngữ tự nhiên như tiền xử lý dữ liệu, phân tích câu, đánh giá mức độ quan trọng của thông tin và tổ chức đầu ra phù hợp.

Đối với tiếng Việt, việc xử lý ngôn ngữ tự nhiên còn gặp thêm nhiều khó khăn do đặc điểm ngôn ngữ như từ ghép, dấu câu, cách biểu đạt linh hoạt và tính phụ thuộc ngữ cảnh cao. Vì vậy, việc nghiên cứu và xây dựng các hệ thống xử lý văn bản tiếng Việt nói chung, cũng như hệ thống tóm tắt văn bản nói riêng, có ý nghĩa quan trọng cả về mặt học thuật lẫn thực tiễn.

2.2. Tổng quan về bài toán tóm tắt văn bản

Tóm tắt văn bản là quá trình rút gọn một văn bản gốc thành một đoạn văn ngắn hơn nhưng vẫn bảo đảm giữ lại được những nội dung quan trọng, các ý chính hoặc thông tin cốt lõi. Mục tiêu của tóm tắt văn bản là giúp người đọc nhanh chóng hiểu được nội dung tổng quát của tài liệu mà không cần phải đọc toàn bộ văn bản

gốc.

Trong bối cảnh lượng dữ liệu văn bản tăng nhanh như hiện nay, bài toán tóm tắt văn bản ngày càng trở nên cần thiết. Người dùng thường xuyên phải tiếp xúc với nhiều loại văn bản khác nhau như tin tức, tài liệu học tập, báo cáo, bài nghiên cứu, email, nội dung website và văn bản hành chính. Trong nhiều trường hợp, thời gian đọc bị hạn chế, nên việc có một bản tóm tắt ngắn gọn và dễ hiểu sẽ hỗ trợ rất nhiều cho quá trình tiếp nhận thông tin.

Bài toán tóm tắt văn bản có thể được thực hiện thủ công hoặc tự động. Tóm tắt thủ công thường do con người đọc, chọn lọc và viết lại những ý quan trọng. Cách làm này cho chất lượng cao nhưng mất nhiều thời gian, công sức và khó áp dụng khi số lượng văn bản lớn. Ngược lại, tóm tắt văn bản tự động cho phép máy tính xử lý và tạo ra bản tóm tắt nhanh hơn, phù hợp với nhu cầu ứng dụng trong thực tế.

Một hệ thống tóm tắt văn bản tốt cần giữ lại được thông tin quan trọng nhất của văn bản, loại bỏ các nội dung dư thừa hoặc lặp lại, tạo ra đầu ra ngắn gọn và dễ đọc, đồng thời vẫn duy trì được mức độ liên kết và hạn chế làm sai lệch ý nghĩa của văn bản gốc.

Tuy nhiên, đây là một bài toán không đơn giản. Để tạo ra một bản tóm tắt chất lượng, hệ thống cần xác định được phần nào của văn bản là quan trọng, phần nào có thể lược bỏ, đồng thời phải cân nhắc đến ngữ cảnh và mối quan hệ giữa các câu. Điều này trở nên khó hơn đối với các văn bản dài, phức tạp hoặc có nhiều thông tin chuyên ngành.

Tóm tắt văn bản hiện được ứng dụng rộng rãi trong thực tế, chẳng hạn để rút gọn tin tức cho người đọc nắm nhanh nội dung, hỗ trợ học tập bằng cách tóm lược tài liệu, tóm tắt báo cáo hoặc văn bản hành chính, và giúp người dùng tìm kiếm thông tin nhanh hơn trong các kho tài liệu lớn hoặc trên nền tảng web.

Với những lợi ích như trên, bài toán tóm tắt văn bản là một chủ đề phù hợp để nghiên cứu và triển khai trong khuôn khổ đồ án công nghệ thông tin.

2.3. Phân loại các phương pháp tóm tắt văn bản

Hiện nay, các phương pháp tóm tắt văn bản thường được chia thành hai nhóm chính là tóm tắt trích xuất và tóm tắt sinh diễn giải. Mỗi phương pháp có nguyên lý hoạt động, ưu điểm và hạn chế riêng.

2.3.1. Phương pháp tóm tắt trích xuất

Tóm tắt trích xuất là phương pháp tạo bản tóm tắt bằng cách lựa chọn trực tiếp những câu, cụm từ hoặc đoạn văn quan trọng từ văn bản gốc, sau đó ghép lại để

tạo thành bản tóm tắt. Phương pháp này không tạo ra câu mới mà chỉ sử dụng lại nội dung đã có sẵn trong văn bản.

Nguyên lý của phương pháp tóm tắt trích xuất thường dựa trên việc đánh giá mức độ quan trọng của từng câu trong văn bản. Trong thực tế, câu có thể được chấm điểm dựa trên tần suất xuất hiện của từ khóa, vị trí trong đoạn hoặc trong bài, mức độ tương đồng với chủ đề chung và mức độ liên kết với các câu khác trong văn bản.

- Sự xuất hiện của các từ quan trọng hoặc cụm từ nổi bật.

Sau khi chấm điểm, hệ thống sẽ chọn ra một số câu có điểm số cao nhất để đưa vào bản tóm tắt. Phương pháp này có ưu điểm là dễ triển khai, tốc độ xử lý nhanh, ít tốn tài nguyên và phù hợp với các hệ thống ở mức cơ bản hoặc trung bình.

Tuy nhiên, tóm tắt trích xuất cũng có một số hạn chế. Do chỉ lấy nguyên câu từ văn bản gốc, bản tóm tắt có thể thiếu tính tự nhiên hoặc chưa thật sự mượt mà về mặt diễn đạt. Ngoài ra, nếu các câu được chọn không liên kết tốt với nhau thì bản tóm tắt có thể bị rời rạc hoặc lặp ý.

Trong khuôn khổ một đồ án xây dựng hệ thống tóm tắtt văn bản, phương pháp trích xuất thường là lựa chọn phù hợp vì dễ hiểu, dễ cài đặt, có khả năng kiểm thử tốt và cho kết quả trực quan.

2.3.2. Phương pháp tóm tắt sinh diễn giải

Tóm tắt sinh diễn giải là phương pháp tạo bản tóm tắt bằng cách sinh ra câu mới dựa trên việc hiểu nội dung văn bản gốc. Không giống như phương pháp trích xuất, phương pháp này không chỉ chọn lại những câu đã có sẵn mà có thể diễn đạt lại nội dung theo cách ngắn gọn và tự nhiên hơn.

Phương pháp tóm tắt sinh diễn giải thường dựa trên các mô hình học máy hoặc học sâu, đặc biệt là các mô hình ngôn ngữ hiện đại. Hệ thống sẽ học cách biểu diễn nội dung văn bản đầu vào và sau đó sinh ra đoạn tóm tắt phù hợp.

Ưu điểm của phương pháp này là có khả năng tạo ra bản tóm tắt ngắn gọn, tự nhiên và linh hoạt hơn. Nội dung đầu ra có thể súc tích hơn so với việc ghép trực tiếp các câu từ văn bản gốc. Đây là hướng tiếp cận gần hơn với cách con người thực hiện tóm tắt.

Tuy nhiên, phương pháp tóm tắt sinh diễn giải có độ phức tạp cao hơn nhiều so với tóm tắt trích xuất. Việc triển khai thường đòi hỏi dữ liệu huấn luyện, tài nguyên tính toán lớn và kiến thức chuyên sâu về mô hình học máy. Ngoài ra, kết quả sinh ra có thể gặp lỗi sai ngữ nghĩa hoặc làm lệch ý nếu mô hình chưa được huấn luyện tốt.

Do những yêu cầu cao về dữ liệu và tính toán, phương pháp này thường phù hợp với các hệ thống lớn hoặc các nghiên cứu chuyên sâu hơn.

2.3.3. So sánh hai phương pháp

Hai phương pháp tóm tắt văn bản nêu trên đều có mục tiêu chung là rút gọn nội dung văn bản nhưng khác nhau về cách thức thực hiện.

Phương pháp tóm tắt trích xuất có ưu điểm là đơn giản, dễ cài đặt, ít tốn tài nguyên và phù hợp với các hệ thống có phạm vi vừa và nhỏ. Tuy nhiên, chất lượng diễn đạt của bản tóm tắt còn phụ thuộc nhiều vào chất lượng các câu được chọn từ văn bản gốc.

Trong khi đó, phương pháp tóm tắt sinh diễn giải có khả năng tạo ra các bản tóm tắt linh hoạt và gần với cách viết của con người hơn. Tuy nhiên, nó yêu cầu kỹ thuật cao hơn, cần dữ liệu huấn luyện lớn và chi phí triển khai cũng cao hơn.

Trong phạm vi của đề tài này, việc lựa chọn hướng tiếp cận phù hợp cần dựa trên tính khả thi, mục tiêu đồ án và mức độ hoàn thiện của hệ thống. Đối với một hệ thống tóm tắt văn bản được xây dựng trong khuôn khổ đồ án học phần, hướng tiếp cận trích xuất thường phù hợp hơn do bảo đảm tính thực tiễn và dễ kiểm chứng.

2.4. Quy trình xử lý văn bản trong hệ thống tóm tắt

Để tạo ra một bản tóm tắt tự động, hệ thống thường phải trải qua nhiều bước xử lý khác nhau. Mỗi bước có vai trò riêng trong việc biến đổi văn bản đầu vào thành dạng phù hợp để phân tích và rút trích nội dung quan trọng.

2.4.1. Thu thập hoặc nhập dữ liệu đầu vào

Đây là bước đầu tiên trong quy trình xử lý. Dữ liệu đầu vào của hệ thống là văn bản mà người dùng cung cấp. Văn bản này có thể là đoạn tin tức, bài viết, đoạn mô tả hoặc nội dung tài liệu ngắn. Chất lượng đầu vào ảnh hưởng trực tiếp đến chất lượng bản tóm tắt đầu ra.

2.4.2. Làm sạch dữ liệu

Văn bản đầu vào thường chứa các thành phần không cần thiết như ký tự đặc biệt, khoảng trắng dư thừa, dấu câu không phù hợp hoặc các ký hiệu gây nhiễu. Vì vậy, cần thực hiện bước làm sạch dữ liệu để loại bỏ các thành phần này, giúp văn bản trở nên chuẩn hơn cho các bước xử lý tiếp theo.

2.4.3. Chuẩn hóa văn bản

Sau khi làm sạch, văn bản được chuẩn hóa về định dạng để bảo đảm tính thống nhất. Các thao tác thường gặp gồm chuyển đổi kiểu chữ khi cần thiết, chuẩn hóa khoảng trắng, chuẩn hóa dấu câu và đưa dữ liệu về một dạng biểu diễn đồng

nhất trước khi tiếp tục các bước xử lý sâu hơn.

2.4.4. Tách câu

Tách câu là bước quan trọng trong nhiều hệ thống tóm tắt trích xuất. Văn bản đầu vào được chia thành các câu riêng biệt để thuận tiện cho việc đánh giá mức độ quan trọng của từng câu. Việc tách câu đúng sẽ giúp hệ thống lựa chọn nội dung chính xác hơn.

2.4.5. Tách từ và xử lý từ

Trong một số hệ thống, văn bản sau khi tách câu sẽ tiếp tục được tách thành các từ hoặc cụm từ có ý nghĩa. Bước này giúp xác định từ khóa, tần suất từ và mối quan hệ giữa các đơn vị ngôn ngữ. Đối với tiếng Việt, đây là bước khá quan trọng do từ có thể gồm nhiều âm tiết cách nhau bằng dấu cách.

2.4.6. Xác định nội dung quan trọng

Sau khi văn bản đã được tiền xử lý, hệ thống tiến hành phân tích để xác định những câu hoặc nội dung quan trọng nhất. Tùy vào phương pháp được sử dụng, việc đánh giá có thể dựa trên tần suất xuất hiện của từ khóa, điểm số của câu, mức độ tương đồng giữa các câu, vị trí câu trong văn bản và khả năng đại diện cho chủ đề chung.

2.4.7. Sinh bản tóm tắt

Từ các nội dung đã được chọn lọc, hệ thống sẽ tạo ra bản tóm tắt cuối cùng. Với hệ thống trích xuất, bước này thường là chọn một số câu tiêu biểu và sắp xếp lại theo thứ tự hợp lý. Kết quả đầu ra cần ngắn gọn, dễ hiểu và phản ánh được nội dung chính của văn bản gốc.

Quy trình xử lý văn bản nêu trên là nền tảng quan trọng để xây dựng một hệ thống tóm tắt văn bản tự động có khả năng hoạt động hiệu quả.

2.5. Một số kỹ thuật và thuật toán thường dùng

Trong bài toán tóm tắt văn bản, có nhiều kỹ thuật và thuật toán đã được nghiên cứu và áp dụng. Dưới đây là một số kỹ thuật phổ biến thường được sử dụng trong các hệ thống tóm tắt tự động.

2.5.1. Tần suất từ

Đây là một trong những kỹ thuật đơn giản và phổ biến nhất. Ý tưởng chính là những từ xuất hiện nhiều lần trong văn bản thường có liên quan đến chủ đề chính. Dựa vào tần suất xuất hiện của từ, hệ thống có thể đánh giá mức độ quan trọng của từng câu. Những câu chứa nhiều từ quan trọng sẽ được ưu tiên lựa chọn vào bản tóm tắt.

Ưu điểm của kỹ thuật này là dễ triển khai, ít tốn tài nguyên và phù hợp với các hệ thống đơn giản. Tuy nhiên, nếu chỉ dựa vào tần suất từ, hệ thống có thể chưa đánh giá đầy đủ ý nghĩa ngữ cảnh.

2.5.2. TF-IDF

TF-IDF là một kỹ thuật dùng để đánh giá mức độ quan trọng của một từ trong văn bản, trong đó TF thể hiện tần suất xuất hiện của từ trong văn bản còn IDF thể hiện mức độ đặc trưng của từ so với toàn bộ tập dữ liệu. Nhờ kết hợp hai thành phần này, TF-IDF giúp làm nổi bật các từ quan trọng và giảm ảnh hưởng của những từ xuất hiện nhiều nhưng ít mang giá trị phân biệt.

TF-IDF giúp làm nổi bật những từ quan trọng hơn, đồng thời giảm ảnh hưởng của các từ xuất hiện nhiều nhưng không mang nhiều ý nghĩa phân biệt. Kỹ thuật này thường được dùng trong các bài toán phân tích văn bản, tìm kiếm thông tin và tóm tắt văn bản.

2.5.3. TextRank

TextRank là một thuật toán dựa trên đồ thị, được phát triển từ ý tưởng của PageRank trong lĩnh vực xếp hạng trang web. Trong bài toán tóm tắt văn bản, mỗi câu trong văn bản được xem như một đỉnh của đồ thị. Mức độ tương đồng giữa các câu được dùng để tạo cạnh nối giữa các đỉnh. Sau đó, hệ thống tính toán điểm quan trọng cho từng câu dựa trên mối liên hệ với các câu còn lại.

Những câu có điểm cao sẽ được chọn để đưa vào bản tóm tắt. TextRank có ưu điểm là không cần dữ liệu huấn luyện, hoạt động khá hiệu quả và được sử dụng nhiều trong các hệ thống tóm tắt trích xuất.

2.5.4. Chấm điểm câu

Đây là kỹ thuật đánh giá độ quan trọng của câu dựa trên một tập tiêu chí nhất định. Trong đó, hệ thống có thể xét đến số lượng từ khóa xuất hiện trong câu, độ dài câu, vị trí câu trong văn bản, mức độ liên hệ với tiêu đề hoặc chủ đề và tần suất của các từ quan trọng. Nhờ vậy, hệ thống dễ lựa chọn được những câu nổi bật hơn để đưa vào bản tóm tắt.

Kỹ thuật chấm điểm câu giúp hệ thống dễ dàng lựa chọn các câu nổi bật nhất để đưa vào bản tóm tắt.

2.5.5. Độ tương đồng giữa các câu

Một số hệ thống sử dụng độ tương đồng để xác định mối quan hệ giữa các câu trong văn bản. Nếu một câu có mức độ tương đồng cao với nhiều câu khác, nó có thể đại diện tốt cho nội dung chung của văn bản. Phương pháp này thường được kết hợp với mô hình đồ thị hoặc các phương pháp xếp hạng câu.

2.5.6. Các mô hình học sâu

Ngoài các kỹ thuật truyền thống, hiện nay nhiều hệ thống tóm tắt văn bản sử dụng các mô hình học sâu như mạng nơ-ron hồi tiếp, mô hình mã hóa – giải mã, Transformer hoặc các mô hình ngôn ngữ lớn. Những mô hình này có thể học được biểu diễn ngữ nghĩa sâu hơn, từ đó tạo ra các bản tóm tắt có chất lượng cao hơn.

Tuy nhiên, việc áp dụng các mô hình học sâu thường đòi hỏi dữ liệu lớn, tài nguyên tính toán mạnh và thời gian huấn luyện dài. Do đó, trong các hệ thống có quy mô nhỏ hoặc đồ án học phần, các kỹ thuật truyền thống vẫn là lựa chọn phù hợp hơn.

2.6. Công nghệ và công cụ sử dụng trong đề tài

Để xây dựng hệ thống tóm tắt văn bản, đề tài sử dụng một số công nghệ và công cụ phù hợp với mục tiêu triển khai, tính khả thi và phạm vi của đồ án.

2.6.1. Ngôn ngữ lập trình Python

Python là ngôn ngữ lập trình chính được sử dụng trong đề tài. Đây là một ngôn ngữ phổ biến, có cú pháp đơn giản, dễ đọc và hỗ trợ tốt cho các bài toán xử lý dữ liệu, trí tuệ nhân tạo và xử lý ngôn ngữ tự nhiên. Python cũng có hệ sinh thái thư viện phong phú, giúp việc xây dựng hệ thống trở nên nhanh chóng và thuận tiện hơn.

2.6.2. Thư viện hỗ trợ xử lý văn bản

Trong quá trình cài đặt hệ thống, các thư viện hỗ trợ xử lý văn bản được sử dụng để phục vụ cho các bước như làm sạch dữ liệu, tách câu, tách từ, tính toán độ quan trọng và sinh ra bản tóm tắt. Việc sử dụng thư viện giúp giảm bớt khối lượng cài đặt thủ công và nâng cao độ tin cậy của chương trình.

2.6.3. Công cụ quản lý phụ thuộc

Đề tài sử dụng tệp cấu hình thư viện để quản lý các gói cần thiết cho hệ thống. Việc quản lý phụ thuộc giúp bảo đảm môi trường chạy ổn định, đồng thời tạo điều kiện thuận lợi cho việc cài đặt lại hệ thống trên máy khác.

2.6.4. Môi trường phát triển

Hệ thống được xây dựng trong môi trường phát triển lập trình hỗ trợ Python. Công cụ phát triển giúp người thực hiện dễ dàng viết mã, kiểm tra lỗi, chạy thử và chỉnh sửa chương trình trong suốt quá trình thực hiện đồ án.

2.6.5. Công cụ kiểm thử và chạy thử

Trong quá trình phát triển, hệ thống được chạy thử với các văn bản mẫu để quan sát đầu ra và đánh giá kết quả. Việc kiểm thử này giúp phát hiện lỗi, điều

chỉnh thuật toán và cải thiện chất lượng của hệ thống trước khi hoàn thiện báo cáo.

Nhìn chung, việc lựa chọn Python cùng các công cụ hỗ trợ phù hợp giúp hệ thống bảo đảm tính khả thi, dễ triển khai, dễ sử dụng và phù hợp với điều kiện thực hiện của sinh viên.

2.7. Kết luận chương

Trong chương này, đề tài đã trình bày các cơ sở lý thuyết làm nền tảng cho việc nghiên cứu và xây dựng hệ thống tóm tắt văn bản. Trước hết, chương đã giới thiệu tổng quan về xử lý ngôn ngữ tự nhiên và vai trò của lĩnh vực này trong các ứng dụng hiện đại. Tiếp theo, bài toán tóm tắt văn bản được phân tích về khái niệm, mục tiêu, ý nghĩa và các ứng dụng thực tế.

Chương cũng đã trình bày hai hướng tiếp cận chính trong tóm tắt văn bản là tóm tắt trích xuất và tóm tắt sinh diễn giải, đồng thời so sánh ưu điểm và hạn chế của từng phương pháp. Bên cạnh đó, các bước cơ bản trong quy trình xử lý văn bản cũng được mô tả nhằm làm rõ cách thức hệ thống chuyển đổi văn bản đầu vào thành bản tóm tắt đầu ra. Cuối cùng, chương đã đề cập đến một số kỹ thuật, thuật toán thường dùng cũng như các công nghệ và công cụ phù hợp cho việc triển khai hệ thống.

Những nội dung lý thuyết được trình bày trong chương này là cơ sở quan trọng để chuyển sang chương tiếp theo, trong đó đề tài sẽ tiến hành phân tích yêu cầu và thiết kế hệ thống tóm tắt văn bản một cách cụ thể hơn.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Khảo sát bài toán và yêu cầu hệ thống

Trong bối cảnh lượng thông tin dưới dạng văn bản ngày càng gia tăng, nhu cầu tiếp cận nội dung một cách nhanh chóng và hiệu quả trở nên rất cần thiết. Người dùng thường phải đọc các đoạn văn, bài báo, tài liệu học tập hoặc nội dung trên Internet có độ dài lớn, trong khi thời gian đọc và chọn lọc thông tin lại có giới hạn. Vì vậy, việc xây dựng một hệ thống có khả năng tự động tóm tắt nội dung văn bản là một giải pháp có ý nghĩa thực tiễn.

Bài toán đặt ra đối với đề tài là xây dựng một hệ thống cho phép người dùng nhập vào một đoạn văn bản, sau đó hệ thống thực hiện xử lý và đưa ra một bản tóm tắt ngắn gọn hơn nhưng vẫn giữ được những ý chính quan trọng của nội dung gốc. Hệ thống cần có cách tổ chức rõ ràng, dễ sử dụng và phù hợp với phạm vi của một đồ án công nghệ thông tin.

Từ yêu cầu thực tế đó, hệ thống được xây dựng theo hướng đơn giản, trực quan và dễ triển khai. Chương trình tập trung vào việc xử lý một văn bản đầu vào tại một thời điểm, thực hiện các bước tiền xử lý, phân tích nội dung, lựa chọn thông tin quan trọng và trả về bản tóm tắt cho người dùng.

Việc khảo sát bài toán cho thấy hệ thống cần đáp ứng được các yêu cầu cơ bản như:

- Có khả năng tiếp nhận văn bản đầu vào từ người dùng.
- Có khả năng xử lý và rút gọn nội dung văn bản.
- Có khả năng hiển thị kết quả tóm tắt rõ ràng, dễ hiểu.
- Có khả năng hoạt động ổn định với các văn bản thông thường.
- Có cấu trúc chương trình thuận tiện cho việc bảo trì và phát triển sau này.

Nhìn chung, đây là một bài toán phù hợp để triển khai trong khuôn khổ đồ án vì vừa có giá trị ứng dụng thực tế, vừa cho phép sinh viên vận dụng các kiến thức về lập trình, xử lý văn bản, phân tích thiết kế hệ thống và xây dựng phần mềm.

3.2. Phân tích yêu cầu chức năng

Yêu cầu chức năng là những chức năng cụ thể mà hệ thống cần thực hiện để đáp ứng nhu cầu sử dụng của người dùng. Đối với hệ thống tóm tắt văn bản, các yêu cầu chức năng chính bao gồm:

3.2.1. Chức năng nhập văn bản đầu vào

Hệ thống phải cho phép người dùng nhập một đoạn văn bản để thực hiện tóm

tắt. Văn bản này là dữ liệu gốc để hệ thống xử lý và sinh ra kết quả đầu ra.

3.2.2. Chức năng kiểm tra dữ liệu đầu vào

Trước khi tiến hành tóm tắt, hệ thống cần kiểm tra dữ liệu người dùng nhập vào. Nếu văn bản rỗng, quá ngắn hoặc không hợp lệ, hệ thống cần có thông báo phù hợp để tránh lỗi trong quá trình xử lý.

3.2.3. Chức năng tiền xử lý văn bản

Sau khi nhận dữ liệu đầu vào, hệ thống cần thực hiện các bước xử lý ban đầu như làm sạch dữ liệu, chuẩn hóa nội dung, loại bỏ ký tự không cần thiết hoặc tách câu để chuẩn bị cho bước tóm tắt.

3.2.4. Chức năng tóm tắt văn bản

Đây là chức năng trọng tâm của hệ thống. Từ văn bản đầu vào đã qua xử lý, hệ thống cần xác định các nội dung quan trọng và sinh ra bản tóm tắt tương ứng.

3.2.5. Chức năng hiển thị kết quả

Sau khi hoàn tất quá trình xử lý, hệ thống phải hiển thị bản tóm tắt cho người dùng một cách rõ ràng, dễ đọc và dễ so sánh với nội dung ban đầu.

3.2.6. Chức năng thao tác lại

Hệ thống nên cho phép người dùng tiếp tục nhập văn bản khác hoặc làm mới nội dung để thực hiện các lần tóm tắt tiếp theo một cách thuận tiện.

Từ các yêu cầu chức năng trên, có thể thấy hệ thống tập trung vào quy trình xử lý khép kín từ tiếp nhận đầu vào, xử lý văn bản, thực hiện tóm tắt đến trả kết quả đầu ra.

3.3. Phân tích yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng, hệ thống còn cần đáp ứng một số yêu cầu phi chức năng để bảo đảm khả năng sử dụng và chất lượng chung của chương trình.

3.3.1. Tính dễ sử dụng

Hệ thống cần có giao diện hoặc cách thức thao tác đơn giản, giúp người dùng dễ dàng nhập nội dung và nhận kết quả mà không cần nhiều kiến thức kỹ thuật.

3.3.2. Hiệu năng xử lý

Thời gian xử lý văn bản cần ở mức hợp lý, đặc biệt với các đoạn văn bản có độ dài vừa phải. Người dùng cần nhận được kết quả trong thời gian ngắn để bảo đảm trải nghiệm sử dụng.

3.3.3. Tính ổn định

Chương trình cần hoạt động ổn định trong các trường hợp sử dụng thông thường, hạn chế lỗi khi người dùng nhập nội dung không đúng hoặc văn bản có định dạng khác nhau.

3.3.4. Tính chính xác tương đối

Do đây là hệ thống tóm tắt tự động nên kết quả có thể chưa đạt mức hoàn hảo như con người, tuy nhiên bản tóm tắt cần phản ánh đúng nội dung chính của văn bản và không làm sai lệch ý nghĩa quá nhiều.

3.3.5. Tính mở rộng

Hệ thống cần có cấu trúc mã nguồn rõ ràng để thuận lợi cho việc nâng cấp sau này, chẳng hạn như thay đổi thuật toán tóm tắt, bổ sung chức năng hoặc cải tiến giao diện.

3.3.6. Tính bảo trì

Mã nguồn nên được tổ chức thành các thành phần hoặc module hợp lý, giúp dễ dàng sửa lỗi, kiểm thử và phát triển tiếp trong tương lai.

Các yêu cầu phi chức năng này góp phần bảo đảm rằng hệ thống không chỉ hoạt động đúng chức năng mà còn đáp ứng được các tiêu chí về chất lượng phần mềm.

3.4. Phân tích tác nhân và ca sử dụng

3.4.1. Tác nhân của hệ thống

Trong phạm vi của đề tài, hệ thống có một tác nhân chính là người dùng.

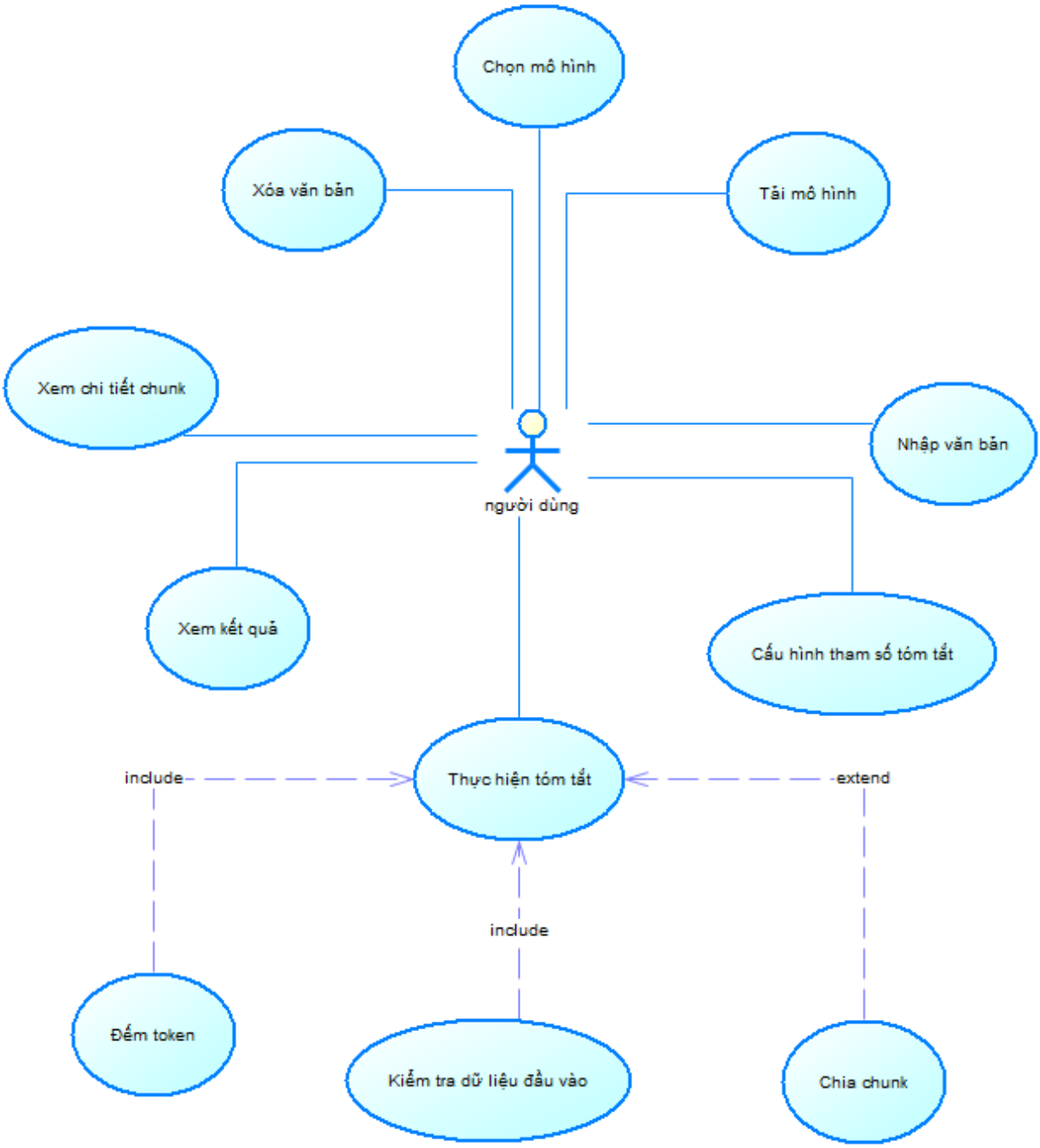
Người dùng là người trực tiếp tương tác với hệ thống, cung cấp văn bản đầu vào, yêu cầu thực hiện tóm tắt và nhận kết quả đầu ra. Người dùng có thể là sinh viên, giảng viên, nhân viên văn phòng hoặc bất kỳ ai có nhu cầu rút gọn nội dung của một văn bản.

3.4.2. Danh sách ca sử dụng

Dựa trên nhu cầu thực tế, hệ thống có các ca sử dụng chính gồm nhập văn bản đầu vào, gửi yêu cầu tóm tắt, nhận kết quả đầu ra, làm mới nội dung để tiếp tục sử dụng và xem thông báo lỗi khi dữ liệu đầu vào không hợp lệ.

3.4.3. Biểu đồ Use Case

Biểu đồ Use Case của hệ thống gồm một tác nhân là Người dùng với các chức năng chính như chọn mô hình, tải mô hình, nhập văn bản, cấu hình tham số tóm tắt, thực hiện tóm tắt, xem kết quả, xem chi tiết chunk và xóa văn bản. Tập chức năng này phản ánh tương đối đầy đủ các thao tác mà người dùng cần thực hiện khi làm việc với ứng dụng.



Hình 3.1. Biểu đồ Use Case của hệ thống tóm tắt văn bản

3.5. Thiết kế luồng xử lý hệ thống

Luồng xử lý của hệ thống được xây dựng theo trình tự từ đầu vào đến đầu ra. Khi người dùng nhập một đoạn văn bản vào hệ thống, chương trình sẽ tiếp nhận dữ liệu và kiểm tra tính hợp lệ của nội dung. Nếu dữ liệu đạt yêu cầu, hệ thống sẽ chuyển sang bước tiền xử lý, bao gồm làm sạch văn bản, chuẩn hóa nội dung và tách câu nếu cần.

Sau bước tiền xử lý, bộ xử lý tóm tắt sẽ tiến hành phân tích văn bản, xác định các câu hoặc nội dung quan trọng dựa trên phương pháp được cài đặt trong hệ thống. Những câu phù hợp sẽ được lựa chọn để tạo thành bản tóm tắt cuối cùng.

Kết quả sau khi xử lý sẽ được gửi trở lại phần giao diện hoặc thành phần điều phối để hiển thị cho người dùng. Trong trường hợp dữ liệu đầu vào không hợp lệ, hệ thống sẽ trả về thông báo lỗi thay vì tiếp tục xử lý.

Luồng xử lý tổng quát của hệ thống có thể mô tả ngắn gọn như sau: người dùng nhập văn bản, hệ thống kiểm tra tính hợp lệ của dữ liệu, thực hiện tiền xử lý, tiến hành tóm tắt, tạo kết quả đầu ra và hiển thị bản tóm tắt cho người dùng. Chuỗi bước này giúp quy trình xử lý rõ ràng, dễ theo dõi và thuận tiện cho việc kiểm soát lỗi.

Luồng xử lý này bảo đảm hệ thống hoạt động theo quy trình logic, rõ ràng và dễ kiểm soát.

3.6. Thiết kế kiến trúc hệ thống

Dựa trên cấu trúc thực tế của dự án, hệ thống được thiết kế theo hướng phân chia thành các thành phần chính nhằm giúp chương trình rõ ràng và dễ quản lý. Các thành phần này phối hợp với nhau để thực hiện quá trình tóm tắt văn bản.

3.6.1. Thành phần giao tiếp và điều phối

Thành phần này chịu trách nhiệm tiếp nhận yêu cầu từ người dùng, nhận văn bản đầu vào, gọi đến bộ xử lý tóm tắt và trả kết quả về cho người dùng. Trong hệ thống, vai trò này được tổ chức trong tệp app.py.

3.6.2. Thành phần xử lý tóm tắt

Đây là thành phần quan trọng nhất của hệ thống, thực hiện các bước xử lý dữ liệu văn bản và áp dụng phương pháp tóm tắt. Thành phần này được tổ chức trong tệp summarizer.py.

3.6.3. Thành phần quản lý phụ thuộc hệ thống

Thành phần này giúp lưu trữ danh sách các thư viện và gói cần thiết để chạy hệ thống. Trong dự án, phần này được thể hiện qua tệp requirements.txt.

3.6.4. Thành phần hướng dẫn sử dụng

Tệp README.md có vai trò mô tả thông tin chung của hệ thống, hướng dẫn cài đặt và sử dụng chương trình. Đây là thành phần hỗ trợ triển khai và bàn giao sản phẩm.

Từ đó, kiến trúc tổng quát của hệ thống có thể mô tả theo luồng Người dùng tương tác với ứng dụng, app.py tiếp nhận và điều phối yêu cầu, summarizer.py đảm nhiệm phần xử lý tóm tắt, sau đó kết quả được trả ngược về giao diện để hiển thị. Kiến trúc này tuy đơn giản nhưng phù hợp với phạm vi đồ án và thuận lợi cho việc bảo trì.

Kiến trúc này tuy đơn giản nhưng phù hợp với phạm vi của đồ án, đồng thời thuận lợi cho việc phát triển và bảo trì sau này.

3.7. Thiết kế các module chức năng

Để hệ thống hoạt động rõ ràng và dễ quản lý, chương trình được chia thành các module chức năng chính như sau:

3.7.1. Module giao diện người dùng

Module giao diện người dùng là nơi tương tác trực tiếp với người dùng, chịu trách nhiệm hiển thị vùng nhập văn bản, nhận dữ liệu đầu vào, hiển thị kết quả tóm tắt và thông báo lỗi hoặc hướng dẫn khi dữ liệu chưa hợp lệ. Module này đóng vai trò cầu nối giữa người dùng và phần xử lý bên trong.

Module này giúp tạo cầu nối giữa người dùng và hệ thống xử lý bên trong.

3.7.2. Module điều khiển hệ thống

Module điều khiển chịu trách nhiệm kết nối giao diện với bộ xử lý tóm tắt. Về chức năng, module này tiếp nhận dữ liệu đầu vào, kiểm tra điều kiện xử lý, gọi các hàm tóm tắt cần thiết, nhận kết quả trả về và chuyển kết quả sang giao diện hiển thị. Trong dự án hiện tại, vai trò này chủ yếu được thể hiện trong app.py.

Trong dự án, module này có thể được thể hiện chủ yếu trong app.py.

3.7.3. Module xử lý tóm tắt văn bản

Đây là module xử lý nghiệp vụ cốt lõi của hệ thống. Module này phụ trách làm sạch dữ liệu văn bản, chuẩn hóa nội dung, tách câu hoặc các thành phần cần thiết, phân tích mức độ quan trọng của các câu, lựa chọn câu tiêu biểu và tạo bản tóm tắt đầu ra. Trong hệ thống, phần này được tổ chức trong summarizer.py.

Trong hệ thống, module này được tổ chức trong summarizer.py.

Việc chia hệ thống thành các module trên giúp mã nguồn rõ ràng hơn, thuận tiện cho việc kiểm thử, sửa lỗi và mở rộng chức năng trong tương lai.

3.8. Thiết kế dữ liệu đầu vào và đầu ra

3.8.1. Dữ liệu đầu vào

Dữ liệu đầu vào của hệ thống là một đoạn văn bản do người dùng cung cấp, có thể là đoạn tin tức, bài viết ngắn, nội dung học tập, đoạn mô tả thông tin hoặc các văn bản thông thường khác. Để hệ thống xử lý ổn định, văn bản đầu vào cần có nội dung rõ ràng, độ dài đủ để tóm tắt, không quá ngắn và nên ở dạng văn bản thuần để thuận lợi cho quá trình tiền xử lý.

3.8.2. Dữ liệu đầu ra

Dữ liệu đầu ra của hệ thống là một bản tóm tắt được sinh ra từ văn bản gốc. Về chất lượng, bản tóm tắt cần ngắn gọn hơn văn bản ban đầu, vẫn giữ được các ý chính quan trọng, dễ đọc, dễ hiểu và hạn chế lặp ý hoặc lan man. Đây là kết quả cuối cùng mà người dùng nhận được sau khi hệ thống hoàn tất quá trình xử lý.

Đầu ra này là kết quả cuối cùng mà người dùng nhận được sau khi hệ thống hoàn tất quá trình xử lý.

3.9. Kết luận chương

Trong chương này, đề tài đã tiến hành phân tích và thiết kế hệ thống tóm tắt văn bản dựa trên yêu cầu thực tế của bài toán. Trước hết, chương đã xác định được các yêu cầu chức năng và phi chức năng mà hệ thống cần đáp ứng. Tiếp theo, tác nhân và các ca sử dụng chính của hệ thống đã được làm rõ nhằm mô tả cách người dùng tương tác với chương trình.

Bên cạnh đó, chương cũng trình bày luồng xử lý tổng quát, kiến trúc hệ thống và cách phân chia các module chức năng chính. Việc xác định rõ vai trò của từng thành phần như giao diện điều phối, bộ xử lý tóm tắt và các tệp hỗ trợ giúp hệ thống có cấu trúc rõ ràng, dễ triển khai và dễ mở rộng. Ngoài ra, dữ liệu đầu vào, đầu ra và các biểu đồ mô tả hoạt động cũng đã được phân tích để hoàn thiện thiết kế tổng thể.

Những nội dung trong chương này là cơ sở quan trọng để chuyển sang chương tiếp theo, trong đó sẽ trình bày cụ thể quá trình xây dựng, cài đặt và triển khai hệ thống tóm tắt văn bản.

CHƯƠNG 4. XÂY DỰNG HỆ THỐNG

4.1. Môi trường và công cụ phát triển

Hệ thống tóm tắt văn bản trong đề tài được xây dựng bằng ngôn ngữ lập trình Python, kết hợp với các thư viện hỗ trợ xử lý ngôn ngữ tự nhiên, học sâu và phát triển giao diện web. Việc lựa chọn Python giúp quá trình cài đặt, tích hợp mô hình và xây dựng giao diện trở nên thuận tiện hơn do hệ sinh thái thư viện phong phú, đặc biệt trong lĩnh vực trí tuệ nhân tạo và xử lý ngôn ngữ tự nhiên.

Theo cấu hình hiện có của dự án, hệ thống sử dụng các thư viện gồm:

- Transformers 4.35.2
- Torch 2.10.0
- Streamlit 1.28.1
- Sentencepiece 0.1.99
- Nltk 3.8.1
- Requests 2.31.0

Hệ thống có thể chạy trên các môi trường phổ biến như Windows, Linux hoặc macOS. Theo hướng dẫn trong tài liệu dự án, ứng dụng yêu cầu Python 3.8 trở lên và có thể tận dụng GPU nếu máy tính hỗ trợ CUDA để tăng tốc quá trình suy luận mô hình. Ngoài ra, hệ thống cũng đã được chuẩn bị để có thể triển khai trên Streamlit Community Cloud. Repo hướng dẫn chạy ứng dụng bằng lệnh streamlit run app.py, đồng thời lưu ý rằng lần chạy đầu tiên cần tải mô hình từ Hugging Face nên cần có kết nối Internet.

Về mặt phát triển, việc tách riêng phần giao diện (app.py) và phần xử lý tóm tắt (summarizer.py) giúp cho hệ thống dễ bảo trì, dễ mở rộng và thuận lợi cho việc kiểm thử từng phần độc lập.

4.2. Cấu trúc mã nguồn chương trình

Dựa trên cấu trúc thực tế của repo, hệ thống được tổ chức tương đối gọn và tập trung vào bốn tệp chính là app.py để xây dựng giao diện và điều phối thao tác người dùng, summarizer.py để cài đặt lớp xử lý tóm tắt, requirements.txt để quản lý thư viện phụ thuộc và README.md để hướng dẫn cài đặt, sử dụng cũng như triển khai hệ thống.

Với cách tổ chức này, hệ thống thể hiện mô hình kiến trúc khá rõ ràng, trong đó app.py đảm nhiệm lớp giao diện và điều phối, summarizer.py phụ trách lớp xử lý nghiệp vụ, còn requirements.txt cùng README.md đóng vai trò cấu hình và tài

liệu. Cấu trúc này phù hợp với một ứng dụng web nhỏ dùng AI vì giúp tách biệt rõ vai trò của từng thành phần.

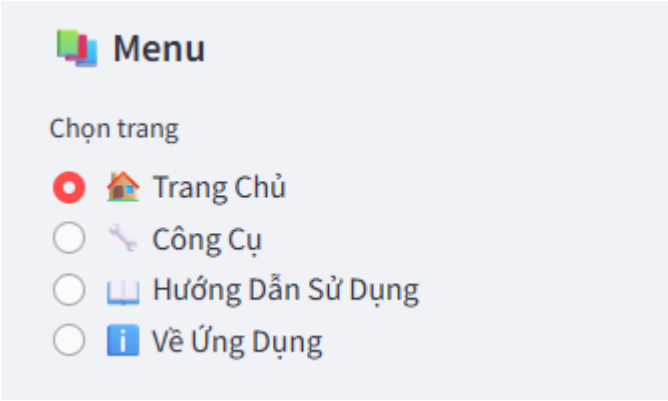
4.3. Cài đặt module giao diện và điều phối

Phần giao diện và điều phối của hệ thống được cài đặt trong tệp app.py. Ứng dụng sử dụng Streamlit để xây dựng giao diện web, nhờ đó người dùng có thể thao tác trực tiếp trên trình duyệt mà không cần cài đặt thêm giao diện desktop riêng.

Ngay từ đầu chương trình, hệ thống định nghĩa hai mô hình mà người dùng có thể lựa chọn là t5-small và facebook/bart-large-cnn. Thông tin mô tả từng mô hình được khai báo trong biến MODEL_OPTIONS để hiển thị giải thích ngay trên giao diện, đồng thời hàm load_summarizer() được gắn cache nhằm giảm việc tải lặp lại không cần thiết.

Thông tin mô tả từng mô hình được khai báo trong biến MODEL_OPTIONS, cho phép hiển thị giải thích ngay trên giao diện. Đồng thời, hàm load_summarizer() được gắn @st.cache_resource để tránh việc tải lại mô hình nhiều lần không cần thiết, từ đó giảm thời gian chờ và tiết kiệm tài nguyên khi ứng dụng được sử dụng lặp lại.

Hệ thống cấu hình trang bằng st.set_page_config() với tiêu đề "Hệ Thống Tóm Tắt Văn Bản", biểu tượng trang là emoji ghi chú, chế độ hiển thị rộng và sidebar mở sẵn. Sau đó, ứng dụng sử dụng st.session_state để lưu trữ các trạng thái làm việc như mô hình đã tải, kết quả tóm tắt, trang hiện tại và văn bản đầu vào. Cơ chế này giúp hệ thống giữ được ngữ cảnh thao tác khi người dùng chuyển trang hoặc thực hiện nhiều lần xử lý liên tiếp.



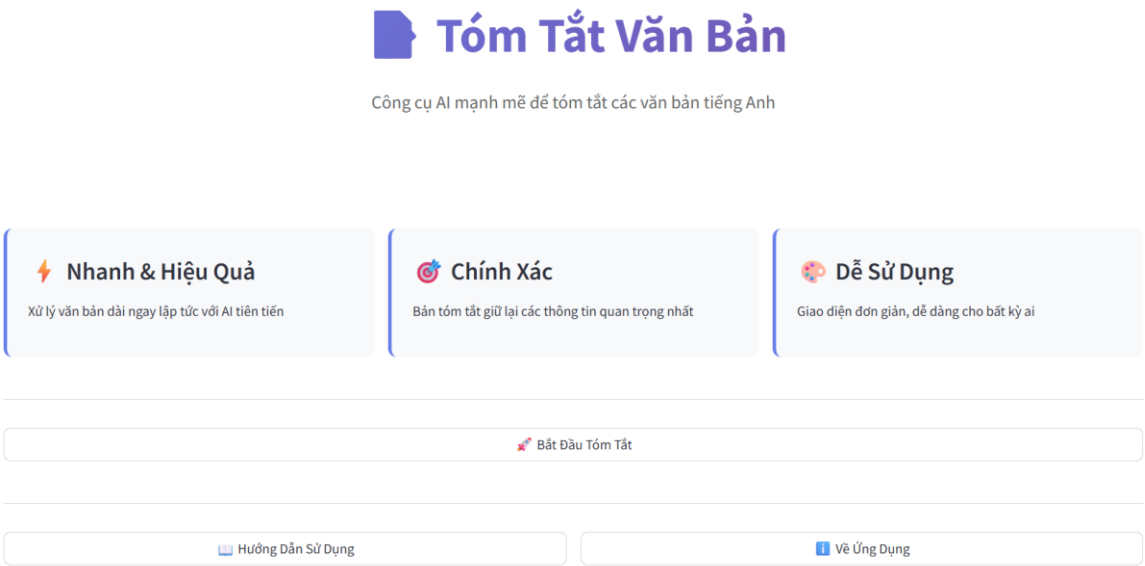
Hình 4.1 menu điều hướng

Về điều hướng, giao diện được chia thành bốn trang là Trang Chủ, Công Cụ, Hướng Dẫn Sử Dụng và Về Ứng Dụng. Sidebar dùng st.radio() để cho phép người dùng chọn nhanh từng trang chức năng, nhờ đó cấu trúc ứng dụng rõ ràng và dễ sử dụng hơn đối với người dùng phổ thông.

Phần sidebar sử dụng st.radio() để cho phép người dùng chọn nhanh từng trang chức năng. Mô hình này giúp ứng dụng có cấu trúc rõ ràng, thân thiện và dễ sử dụng hơn đối với người dùng phổ thông.

4.4. Cài đặt giao diện các chức năng chính

4.4.1. Trang chủ

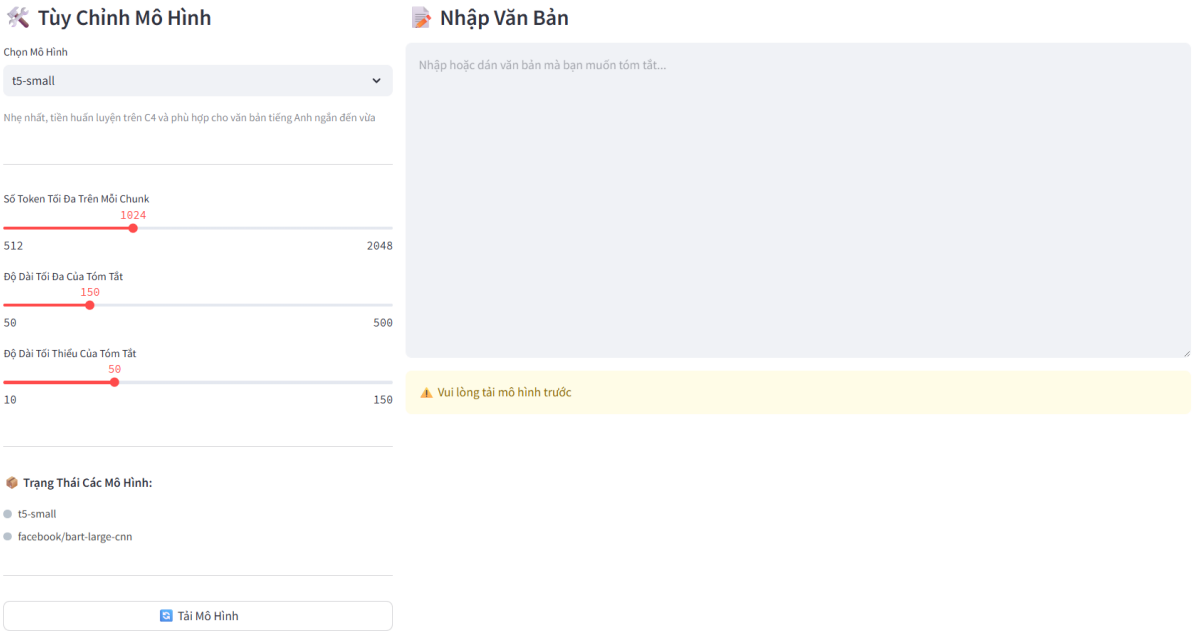


Hình 4.2 giao diện trang chủ

Trang chủ được thiết kế nhằm giới thiệu nhanh về hệ thống và nhấn mạnh ba đặc điểm chính là tốc độ xử lý tương đối nhanh, khả năng tạo bản tóm tắt bám ý chính và giao diện dễ sử dụng. Ngoài ra, trang này còn bố trí các nút điều hướng nhanh đến trang công cụ, trang hướng dẫn và trang thông tin ứng dụng để hỗ trợ người dùng làm quen nhanh hơn.

Ngoài ra, trang chủ còn có các nút điều hướng nhanh đến trang công cụ, trang hướng dẫn sử dụng và trang thông tin về ứng dụng. Việc tổ chức này giúp nâng cao trải nghiệm người dùng và làm cho ứng dụng mang tính hoàn chỉnh hơn so với một giao diện chỉ có duy nhất khung nhập văn bản.

4.4.2. Trang công cụ

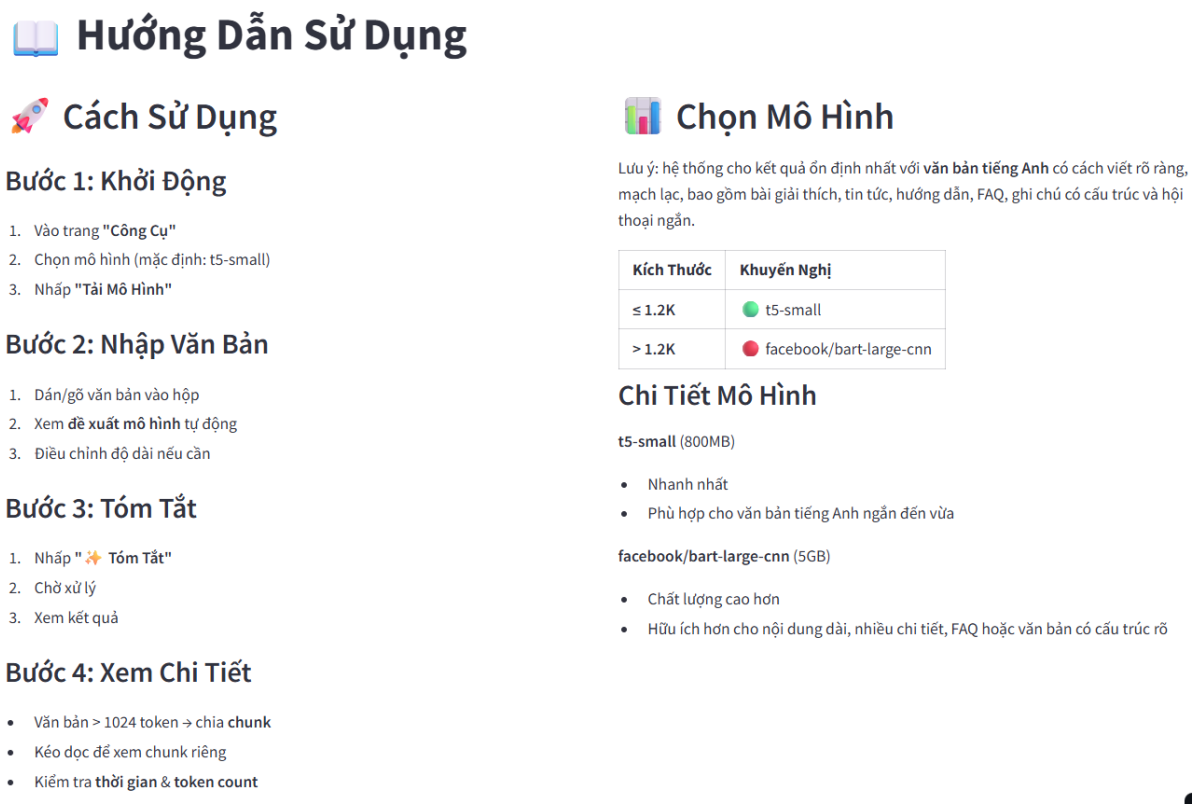


hình 4.3 giao diện trang công cụ

Đây là trang quan trọng nhất của hệ thống. Giao diện được chia thành hai cột, trong đó cột bên trái dùng để cấu hình mô hình và tham số còn cột bên phải dùng để nhập văn bản và hiển thị kết quả. Trên trang này, người dùng có thể chọn mô hình tóm tắt, điều chỉnh số token tối đa trên mỗi chunk, thiết lập độ dài tối đa và tối thiểu của bản tóm tắt, tải mô hình, nhập văn bản, thực hiện tóm tắt và xóa nội dung để làm mới giao diện. Hệ thống cũng hiển thị trạng thái tải mô hình, số token đầu vào và gợi ý mô hình phù hợp theo độ dài văn bản.

Hệ thống còn hiển thị trạng thái tải mô hình, số token của văn bản đầu vào và gợi ý mô hình phù hợp theo độ dài văn bản. Đây là điểm mạnh đáng chú ý của repo vì không chỉ đơn thuần gọi mô hình mà còn hỗ trợ người dùng ra quyết định sử dụng mô hình hợp lý hơn.

4.4.3. Trang hướng dẫn sử dụng



📖 Hướng Dẫn Sử Dụng

🚀 Cách Sử Dụng

Bước 1: Khởi Động

1. Vào trang **"Công Cụ"**
2. Chọn mô hình (mặc định: t5-small)
3. Nhấp **"Tải Mô Hình"**

Bước 2: Nhập Văn Bản

1. Dán/gõ văn bản vào hộp
2. Xem **đề xuất mô hình** tự động
3. Điều chỉnh độ dài nếu cần

Bước 3: Tóm Tắt

1. Nhấp **"✨ Tóm Tắt"**
2. Chờ xử lý
3. Xem kết quả

Bước 4: Xem Chi Tiết

- Văn bản > 1024 token → chia **chunk**
- Kéo dọc để xem chunk riêng
- Kiểm tra **thời gian** & **token count**

📊 Chọn Mô Hình

Lưu ý: hệ thống cho kết quả ổn định nhất với **văn bản tiếng Anh** có cách viết rõ ràng, mạch lạc, bao gồm bài giải thích, tin tức, hướng dẫn, FAQ, ghi chú có cấu trúc và hội thoại ngắn.

Kích Thước	Khuyến Nghị
≤ 1.2K	🟢 t5-small
> 1.2K	🔴 facebook/bart-large-cnn

Chi Tiết Mô Hình

t5-small (800MB)

- Nhanh nhất
- Phù hợp cho văn bản tiếng Anh ngắn đến vừa

facebook/bart-large-cnn (5GB)

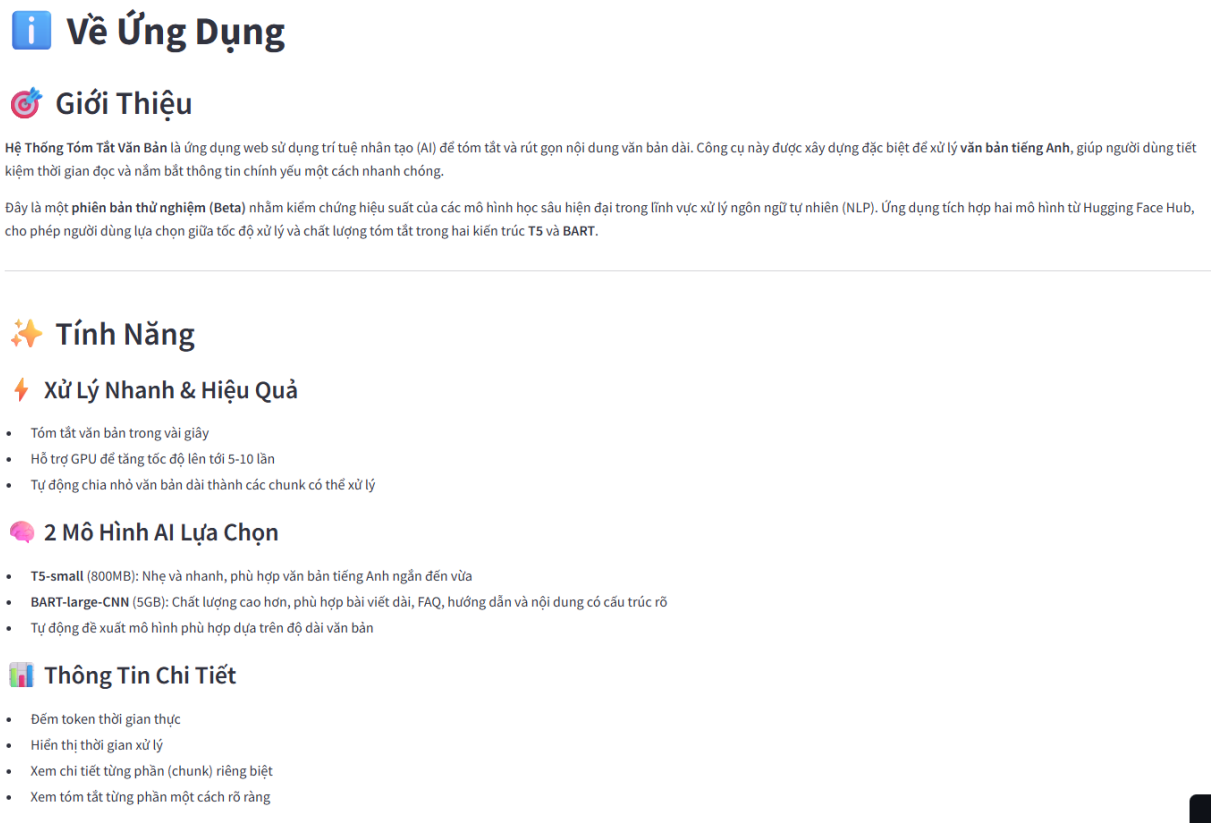
- Chất lượng cao hơn
- Hữu ích hơn cho nội dung dài, nhiều chi tiết, FAQ hoặc văn bản có cấu trúc rõ

Hình 4.4 giao diện trang hướng dẫn sử dụng

Trang hướng dẫn sử dụng mô tả quy trình thao tác theo trình tự từ vào trang công cụ, chọn mô hình, tải mô hình, nhập văn bản, thực hiện tóm tắt đến xem kết quả và chi tiết các chunk nếu có. Việc bổ sung trang hướng dẫn giúp ứng dụng phù hợp hơn với những người dùng không chuyên về kỹ thuật.

Việc bổ sung trang hướng dẫn giúp ứng dụng phù hợp hơn với người dùng không chuyên về kỹ thuật.

4.4.4 Trang về ứng dụng



ℹ️ Về Ứng Dụng

🎯 Giới Thiệu

Hệ Thống Tóm Tắt Văn Bản là ứng dụng web sử dụng trí tuệ nhân tạo (AI) để tóm tắt và rút gọn nội dung văn bản dài. Công cụ này được xây dựng đặc biệt để xử lý **văn bản tiếng Anh**, giúp người dùng tiết kiệm thời gian đọc và nắm bắt thông tin chính yếu một cách nhanh chóng.

Đây là một **phiên bản thử nghiệm (Beta)** nhằm kiểm chứng hiệu suất của các mô hình học sâu hiện đại trong lĩnh vực xử lý ngôn ngữ tự nhiên (NLP). Ứng dụng tích hợp hai mô hình từ Hugging Face Hub, cho phép người dùng lựa chọn giữa tốc độ xử lý và chất lượng tóm tắt trong hai kiến trúc **T5** và **BART**.

✨ Tính Năng

⚡ Xử Lý Nhanh & Hiệu Quả

- Tóm tắt văn bản trong vài giây
- Hỗ trợ GPU để tăng tốc độ lên tới 5-10 lần
- Tự động chia nhỏ văn bản dài thành các chunk có thể xử lý

🧠 2 Mô Hình AI Lựa Chọn

- **T5-small** (800MB): Nhẹ và nhanh, phù hợp văn bản tiếng Anh ngắn đến vừa
- **BART-large-CNN** (5GB): Chất lượng cao hơn, phù hợp bài viết dài, FAQ, hướng dẫn và nội dung có cấu trúc rõ
- Tự động đề xuất mô hình phù hợp dựa trên độ dài văn bản

📊 Thông Tin Chi Tiết

- Đếm token thời gian thực
- Hiển thị thời gian xử lý
- Xem chi tiết từng phần (chunk) riêng biệt
- Xem tóm tắt từng phần một cách rõ ràng

Hình 4.5 giao diện trang về ứng dụng

Trang "Về Ứng Dụng" đóng vai trò như phần giới thiệu kỹ thuật của hệ thống. Nội dung trang cho biết ứng dụng sử dụng AI để tóm tắt văn bản, giao diện được thiết kế bằng tiếng Việt, đầu vào tối ưu là văn bản tiếng Anh, mô hình sử dụng thuộc hai họ T5 và BART, hệ thống hỗ trợ CPU hoặc GPU, có cơ chế chia chunk và kèm theo thông tin phần cứng cũng như ước lượng hiệu năng khi vận hành.

4.5. Cài đặt module xử lý tóm tắt văn bản

Phần xử lý tóm tắt văn bản được cài đặt trong summarizer.py, trong đó thành phần trung tâm là lớp TextSummarizer. Lớp này đảm nhiệm việc nạp tokenizer và mô hình pretrained, xác định thiết bị chạy là CPU hay GPU, đếm token, tách câu, chia văn bản dài thành các chunk, sinh bản tóm tắt cho từng phần và ghép chúng lại thành kết quả cuối cùng.

Ở phiên bản mã nguồn hiện tại, hệ thống không phụ thuộc cứng vào punkt của NLTK trong mọi trường hợp mà đã có cơ chế dự phòng khi dữ liệu tách câu không sẵn có. Nhờ vậy, chức năng xử lý câu vẫn có thể hoạt động ổn định hơn ở các môi trường triển khai khác nhau.

4.5.1. Đếm token

Hệ thống sử dụng tokenizer của chính mô hình được chọn để đếm số token

trong văn bản. Đây là cách làm hợp lý hơn so với việc đếm ký tự hay số từ đơn thuần, vì giới hạn đầu vào của mô hình sinh tóm tắt thường được xác định theo token.

4.5.2. Tách câu

Hàm split_into_sentences() sử dụng nltk.sent_tokenize() để chia văn bản thành danh sách các câu. Bước này là nền tảng để thực hiện chia chunk theo câu, giúp các chunk giữ được tính tự nhiên và ít làm vỡ ngữ cảnh hơn so với chia cắt tùy ý theo ký tự.

4.5.3. Chia văn bản thành chunk

Nếu văn bản dài hơn giới hạn token mà mô hình có thể xử lý, hệ thống dùng hàm chunk_text_by_tokens() để chia nội dung thành nhiều đoạn nhỏ bằng cách cộng dồn token theo từng câu. Nếu thêm câu mới mà chưa vượt ngưỡng thì câu đó được giữ trong chunk hiện tại; nếu vượt ngưỡng thì hệ thống tạo chunk mới; còn với câu quá dài, câu đó có thể trở thành một chunk riêng. Đây là cơ chế quan trọng giúp hệ thống xử lý đầu vào dài mà không lỗi vượt giới hạn mô hình.

Đây là cơ chế quan trọng giúp repo xử lý được văn bản dài mà không bị lỗi vượt ngưỡng đầu vào của mô hình

4.5.4. Tóm tắt từng đoạn văn bản

Hàm summarize_single() dùng tokenizer để mã hóa văn bản thành tensor, sau đó gọi model.generate() để sinh bản tóm tắt. Trong quá trình này, hệ thống sử dụng các tham số chính như max_length, min_length, num_beams = 4 và early_stopping = True để điều khiển độ dài cũng như chất lượng đầu ra. Việc dùng beam search cho thấy hệ thống không sinh văn bản theo cách hoàn toàn ngẫu nhiên mà có bước tìm kiếm kết quả tốt hơn.

Các tham số này giúp điều khiển độ dài và chất lượng của bản tóm tắt. Việc dùng beam search với num_beams=4 cho thấy hệ thống không sinh tóm tắt theo cách hoàn toàn ngẫu nhiên mà có bước tìm kiếm đầu ra tốt hơn.

4.6. Mô tả chi tiết luồng xử lý chương trình

Luồng xử lý thực tế của hệ thống có thể mô tả theo các bước sau:

Bước 1: Khởi động ứng dụng

Người dùng chạy hệ thống bằng lệnh streamlit run app.py. Ứng dụng khởi tạo giao diện, session state và các trang chức năng theo định nghĩa trong app.py.

Bước 2: Chọn mô hình và tải mô hình

Tại trang "Công Cụ", người dùng chọn một trong hai mô hình: t5-small,

facebook/bart-large-cnn. Sau đó nhấn nút "Tải Mô Hình". Hệ thống gọi load_summarizer() để khởi tạo đối tượng TextSummarizer, nạp tokenizer và mô hình tương ứng vào bộ nhớ.

Bước 3: Nhập văn bản

Người dùng dán văn bản vào ô nhập liệu. Ứng dụng dùng hàm count_tokens() để tính số token và hiển thị trực tiếp trên giao diện. Đồng thời hệ thống đưa ra gợi ý mô hình phù hợp dựa theo độ dài văn bản.

Bước 4: Cấu hình tham số tóm tắt

Ở bước cấu hình tham số, người dùng chọn số token tối đa trên mỗi chunk, độ dài tối đa của bản tóm tắt và độ dài tối thiểu của bản tóm tắt. Các giá trị này sẽ được truyền vào hàm summarize() ở thời điểm thực hiện tóm tắt.

Các tham số này được truyền vào hàm summarize() khi người dùng thực hiện tóm tắt.

Bước 5: Thực hiện tóm tắt

Khi nhấn nút "Tóm Tắt Văn Bản", hệ thống trước hết kiểm tra văn bản có rỗng hay không và xác nhận min_summary_length không lớn hơn max_summary_length. Nếu điều kiện hợp lệ, hàm summarize() sẽ đếm token của toàn bộ văn bản, quyết định tóm tắt trực tiếp hay chia chunk, tóm tắt từng phần, ghép các kết quả trung gian và tiếp tục rút gọn lần cuối nếu phần ghép vẫn còn quá dài.

Bước 6: Hiển thị kết quả

Sau khi xử lý xong, hệ thống hiển thị bản tóm tắt cuối cùng, số token của văn bản gốc, thời gian xử lý và trạng thái có chia chunk hay không. Nếu đầu vào được tách thành nhiều chunk, ứng dụng còn cho phép xem chi tiết từng chunk gốc và bản tóm tắt tương ứng dưới dạng vùng mở rộng, điều này rất hữu ích cho mục đích minh họa và đánh giá trong báo cáo.

Nếu văn bản được chia thành nhiều chunk, ứng dụng còn hiển thị chi tiết từng chunk gốc và bản tóm tắt tương ứng dưới dạng các vùng mở rộng (expander). Đây là một ưu điểm tốt cho mục đích minh họa và đánh giá hệ thống trong báo cáo đồ án.

4.7. Kết quả đầu ra và cấu trúc dữ liệu trả về

Khác với nhiều hệ thống chỉ trả về một chuỗi văn bản tóm tắt, mã nguồn hiện tại thiết kế đầu ra theo dạng dictionary và cung cấp khá nhiều thông tin phục vụ hiển thị lẫn phân tích. Cụ thể, hàm summarize() trả về các trường như original_text, token_count, chunks, chunk_summaries, final_summary, needs_chunking và

num_chunks khi có chia đoạn. Thiết kế này giúp giao diện không chỉ hiển thị kết quả cuối mà còn theo dõi được toàn bộ quá trình xử lý trung gian.

Thiết kế này giúp giao diện có thể hiển thị không chỉ kết quả cuối cùng mà còn cho phép theo dõi toàn bộ quá trình xử lý trung gian. Đây là điểm rất nên đưa vào báo cáo vì thể hiện hệ thống được thiết kế có tổ chức, có khả năng giải thích kết quả thay vì chỉ "ra đáp án".

4.8. Một số đặc điểm kỹ thuật nổi bật của hệ thống

Từ repo thực tế, có thể rút ra một số đặc điểm kỹ thuật nổi bật của hệ thống như sau:

4.8.1. Hỗ trợ nhiều mô hình pretrained

Hệ thống không cố định một mô hình duy nhất mà cho phép người dùng chọn giữa hai mô hình khác nhau thuộc hai họ kiến trúc Transformer là T5 và BART. Điều này làm tăng tính linh hoạt và cho phép so sánh giữa tốc độ và chất lượng.

4.8.2. Hỗ trợ xử lý văn bản dài bằng chunking

Đây là đặc điểm kỹ thuật nổi bật nhất của hệ thống. Với các mô hình Transformer, đầu vào thường bị giới hạn số token nên hệ thống khắc phục bằng cách chia văn bản thành các chunk theo câu, tóm tắt từng chunk, ghép các bản tóm tắt lại và rút gọn thêm một lần cuối nếu cần. Cách làm này giúp ứng dụng xử lý được văn bản dài hơn khả năng đầu vào trực tiếp của mô hình.

Cách làm này giúp ứng dụng xử lý được văn bản dài hơn khả năng đầu vào trực tiếp của mô hình.

4.8.3. Tự động chọn CPU hoặc GPU

Trong summarizer.py, hệ thống dùng biểu thức torch.device("cuda" if torch.cuda.is_available() else "cpu") để tự động chọn thiết bị chạy. Nghĩa là nếu máy có GPU hỗ trợ CUDA thì mô hình sẽ được đưa lên GPU, còn nếu không có thì chương trình sẽ chuyển về CPU.

Tức là nếu máy có GPU hỗ trợ CUDA thì mô hình sẽ được chạy trên GPU; nếu không thì fallback về CPU. Điều này giúp ứng dụng thích nghi với nhiều môi trường triển khai khác nhau.

4.8.4. Tối ưu trải nghiệm trên Streamlit Cloud

Theo README.md, dự án đã được điều chỉnh để phù hợp hơn với Streamlit Community Cloud, bao gồm:

- dùng st.cache_resource để tránh tải lại model,

• chọn t5-small làm mô hình mặc định an toàn hơn,

• điều chỉnh dependency để build ổn định hơn trên Linux và phiên bản Python mặc định.

4.8.5. Giao diện tiếng Việt, xử lý văn bản tiếng Anh

Đây là điểm cần viết đúng trong báo cáo để tránh sai lệch. Phần README và trang About đều mô tả hệ thống là công cụ AI tóm tắt văn bản tiếng Anh, trong khi giao diện người dùng được xây dựng 100% tiếng Việt. Nghĩa là đối tượng sử dụng hướng tới người dùng Việt Nam nhưng đầu vào tối ưu là tiếng Anh.

4.9. Hướng dẫn cài đặt và chạy hệ thống

Theo hướng dẫn trong README.md, quy trình cài đặt cơ bản của hệ thống gồm tạo môi trường ảo Python, kích hoạt môi trường, cài đặt thư viện từ requirements.txt và chạy ứng dụng bằng lệnh streamlit run app.py. Ở phiên bản mã nguồn hiện tại, hệ thống đã có cơ chế tách câu dự phòng nên không còn phụ thuộc bắt buộc vào dữ liệu punkt của NLTK trong mọi trường hợp. Khi khởi động lần đầu, mô hình sẽ được tải từ Hugging Face nên thời gian chờ có thể lâu hơn bình thường; README cũng lưu ý dung lượng mô hình có thể dao động từ khoảng 800 MB đến 5 GB tùy theo lựa chọn mô hình.

4.10. Đánh giá việc cài đặt theo repo thực tế

Dựa trên mã nguồn hiện có, có thể thấy hệ thống đã được cài đặt ở mức khá hoàn chỉnh đối với phạm vi một đồ án môn học. Ứng dụng không chỉ dừng ở mức viết hàm tóm tắt mà đã có giao diện người dùng bằng Streamlit, cơ chế quản lý trạng thái thao tác, chức năng lựa chọn mô hình, cấu hình tham số, xử lý lỗi đầu vào, hiển thị kết quả chi tiết và khả năng chạy cục bộ hoặc triển khai trên cloud. Cách tổ chức này cho thấy sản phẩm đã hình thành một ứng dụng có thể sử dụng và trình bày thực tế.

So với một demo tóm tắt văn bản đơn thuần, mã nguồn hiện tại thể hiện tương đối rõ ba lớp chính của hệ thống: lớp giao diện người dùng trong app.py, lớp xử lý nghiệp vụ trong summarizer.py và lớp cấu hình phụ thuộc đi kèm trong requirements.txt cùng tài liệu README. Đây là điểm mạnh quan trọng vì báo cáo có thể chứng minh sản phẩm không chỉ có thuật toán mà còn có khả năng triển khai để người dùng thao tác trực tiếp.

4.11. Kết luận chương

Trong chương này, đề tài đã trình bày quá trình xây dựng hệ thống tóm tắt văn bản bám sát theo mã nguồn thực tế của repo Kain3229/hethongtomtatvanban. Hệ thống được phát triển bằng Python, sử dụng Streamlit để xây dựng giao diện web và

kết hợp các mô hình Transformer pretrained từ Hugging Face để thực hiện tác vụ tóm tắt văn bản.

Chương đã mô tả chi tiết cấu trúc dự án, vai trò của từng tệp mã nguồn, cách tổ chức giao diện, cơ chế điều phối thao tác người dùng, cũng như cách cài đặt module xử lý tóm tắt trong summarizer.py. Đặc biệt, hệ thống có hỗ trợ chia nhỏ văn bản theo token để xử lý các đầu vào dài, đây là điểm kỹ thuật nổi bật và có giá trị thực tiễn cao. Ngoài ra, ứng dụng còn cho phép lựa chọn nhiều mô hình, hiển thị số token, thời gian xử lý và kết quả chi tiết cho từng chunk.

Từ những nội dung đã trình bày, có thể thấy hệ thống không chỉ đáp ứng được yêu cầu xây dựng một ứng dụng tóm tắt văn bản tự động mà còn thể hiện được tính hoàn chỉnh về mặt phần mềm. Đây là cơ sở để chuyển sang chương tiếp theo, trong đó sẽ trình bày kết quả thực nghiệm và đánh giá hiệu quả hoạt động của hệ thống.

CHƯƠNG 5. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1. Mục tiêu thực nghiệm

Sau khi hoàn thành việc xây dựng hệ thống, bước tiếp theo là tiến hành thực nghiệm để kiểm tra khả năng hoạt động thực tế của chương trình. Mục tiêu của quá trình thực nghiệm không chỉ nhằm xác nhận rằng hệ thống có thể chạy đúng về mặt chức năng, mà còn giúp đánh giá mức độ hiệu quả của giải pháp đã triển khai trên repo hiện tại.

Cụ thể, quá trình thực nghiệm của đề tài hướng đến việc kiểm tra khả năng khởi chạy ổn định của ứng dụng Streamlit, khả năng tải và sử dụng hai mô hình tóm tắt đã tích hợp là t5-small và facebook/bart-large-cnn, khả năng xử lý văn bản có độ dài khác nhau thông qua cơ chế chunking, cũng như ảnh hưởng của từng mô hình đến thời gian xử lý, chất lượng đầu ra và mức sử dụng tài nguyên.

Do mã nguồn hiện tại chưa tích hợp trực tiếp ROUGE hoặc BLEU trong giao diện, phần đánh giá trong chương này kết hợp hai nguồn dữ liệu: quan sát thực nghiệm khi chạy hệ thống và benchmark nội bộ được lưu trong benchmark_report.json. Cách làm này giúp báo cáo giảm bớt tính cảm tính vì ngoài nhận xét định tính còn có các số liệu về reference score, fact score, support score và accuracy tổng hợp để đối chiếu giữa hai mô hình.

Việc lựa chọn hướng đánh giá này là phù hợp với phạm vi của đồ án và phản ánh đúng trạng thái triển khai thực tế của hệ thống.

5.2. Dữ liệu thực nghiệm

Theo tài liệu trong repo và nội dung hiển thị trong giao diện ứng dụng, hệ thống hiện được xây dựng để xử lý văn bản tiếng Anh, trong khi giao diện người dùng được thiết kế hoàn toàn bằng tiếng Việt. Vì vậy, dữ liệu thực nghiệm cần được lựa chọn theo đúng định hướng này để bảo đảm kết quả phản ánh đúng khả năng của hệ thống.

Các văn bản dùng để kiểm thử hệ thống có thể được chia thành các nhóm sau:

5.2.1. Văn bản ngắn

Nhóm văn bản ngắn gồm các đầu vào có số lượng token tương đối thấp, thường dưới khoảng 1000 token. Với nhóm này, hệ thống có thể tóm tắt trực tiếp mà không cần chia chunk; các ví dụ phù hợp là tin tức ngắn, mô tả sản phẩm, đoạn giới thiệu chủ đề học tập hoặc nội dung thông tin ngắn trên website.

5.2.2. Văn bản trung bình

Nhóm văn bản trung bình có độ dài khoảng 1000 đến 3000 token. Đây là loại

dữ liệu phù hợp để đánh giá rõ hơn cơ chế đếm token và gợi ý mô hình của hệ thống; các ví dụ đại diện gồm bài báo tiếng Anh có độ dài trung bình, bài viết blog, nội dung tổng hợp thông tin hoặc bài giới thiệu chuyên đề ngắn.

5.2.3. Văn bản dài

Nhóm văn bản dài gồm các đầu vào vượt ngưỡng xử lý trực tiếp của mô hình, vì vậy hệ thống phải kích hoạt cơ chế chia chunk trước khi sinh bản tóm tắt cuối cùng. Đây là nhóm dữ liệu quan trọng nhất để kiểm tra hiệu quả thiết kế trong summarizer.py; các ví dụ phù hợp là tài liệu học tập dài, bài viết phân tích chuyên sâu hoặc văn bản tổng hợp nhiều đoạn thông tin.

5.2.4. Dữ liệu kiểm thử lỗi

Ngoài các văn bản hợp lệ, hệ thống cũng được thử với một số trường hợp biên để đánh giá tính ổn định, chẳng hạn như văn bản rỗng, văn bản quá ngắn, cấu hình min_summary_length lớn hơn max_summary_length hoặc trường hợp người dùng yêu cầu tóm tắt khi mô hình chưa được tải. Các tình huống này giúp kiểm tra khả năng kiểm soát lỗi của giao diện và module xử lý.

5.3. Kịch bản kiểm thử

Để kiểm tra đầy đủ các chức năng đã được xây dựng trong repo, quá trình thực nghiệm được tổ chức theo các kịch bản cụ thể như sau:

5.3.1. Kiểm thử khởi động hệ thống

Kịch bản kiểm thử khởi động nhằm xác nhận ứng dụng có thể chạy thành công bằng lệnh streamlit run app.py, đồng thời giao diện hiển thị đúng các trang chức năng và sidebar điều hướng hoạt động ổn định. Kết quả mong đợi là người dùng có thể truy cập đầy đủ bốn trang chính gồm Trang Chủ, Công Cụ, Hướng Dẫn Sử Dụng và Về Ứng Dụng mà không phát sinh lỗi khi chuyển trang.

5.3.2. Kiểm thử tải mô hình

Kịch bản tải mô hình được thực hiện bằng cách chọn một mô hình trong danh sách, nhấn nút tải và quan sát thanh tiến trình. Hệ thống đạt yêu cầu khi mô hình được tải thành công, trạng thái chuyển sang sẵn sàng và người dùng có thể tiếp tục thực hiện thao tác tóm tắt mà không cần khởi động lại ứng dụng.

5.3.3. Kiểm thử với văn bản ngắn

Với văn bản ngắn có số token nhỏ hơn giới hạn max_tokens, hệ thống được kỳ vọng sẽ tóm tắt trực tiếp mà không cần chia chunk. Trạng thái mong đợi là token được đếm đúng, needs_chunking nhận giá trị False, bản tóm tắt được sinh trong một lần xử lý và thời gian phản hồi ở mức ngắn.

5.3.4. Kiểm thử với văn bản dài

Kịch bản kiểm thử văn bản dài nhằm xác minh cơ chế chunking của hệ thống. Người thực hiện nhập văn bản vượt ngưỡng token, chọn mô hình phù hợp rồi yêu cầu tóm tắt. Kết quả đạt yêu cầu khi hệ thống phát hiện đầu vào quá dài, chia văn bản thành nhiều chunk, tóm tắt từng phần riêng biệt và ghép lại thành final_summary, đồng thời giao diện hiển thị rõ số lượng chunk cùng nội dung tóm tắt tương ứng của từng phần.

5.3.5. Kiểm thử cấu hình tham số

Kịch bản kiểm thử tham số tập trung vào ba nhóm cấu hình là số token tối đa trên mỗi chunk, độ dài tối đa của bản tóm tắt và độ dài tối thiểu của bản tóm tắt. Hệ thống đạt yêu cầu khi nhận đúng giá trị người dùng chọn, cảnh báo hợp lệ trong trường hợp min_summary_length lớn hơn max_summary_length và cho đầu ra thay đổi tương ứng với cấu hình mới.

5.3.6. Kiểm thử tính năng xóa văn bản

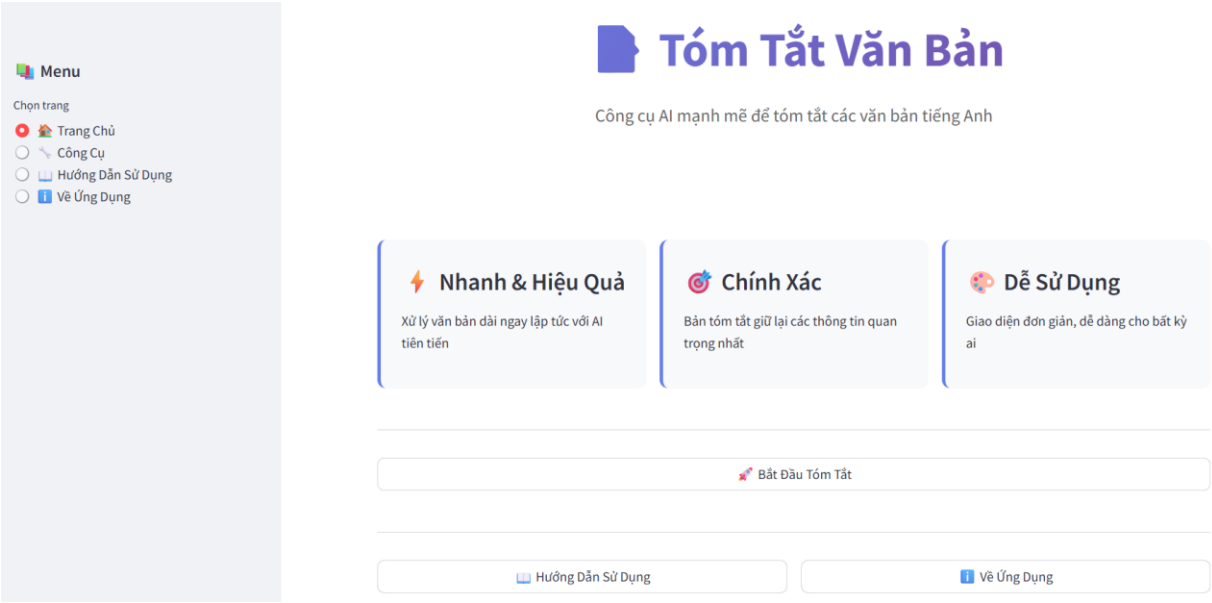
Ở chức năng xóa văn bản, kết quả mong đợi là nội dung đầu vào được làm trống hoàn toàn, kết quả cũ bị xóa khỏi giao diện và người dùng có thể tiếp tục nhập dữ liệu mới để xử lý mà không cần tải lại trang.

5.3.7. Kiểm thử trên CPU/GPU

Trong summarizer.py, hệ thống tự động ưu tiên CUDA khi môi trường có GPU; vì vậy kịch bản này được dùng để đánh giá khả năng thích nghi của chương trình với từng cấu hình phần cứng. Hệ thống được xem là đạt yêu cầu khi có thể chạy trên GPU nếu khả dụng, chuyển sang CPU khi không có CUDA và vẫn giữ được tính đúng đắn của đầu ra trong cả hai trường hợp.

5.4. Kết quả thực nghiệm

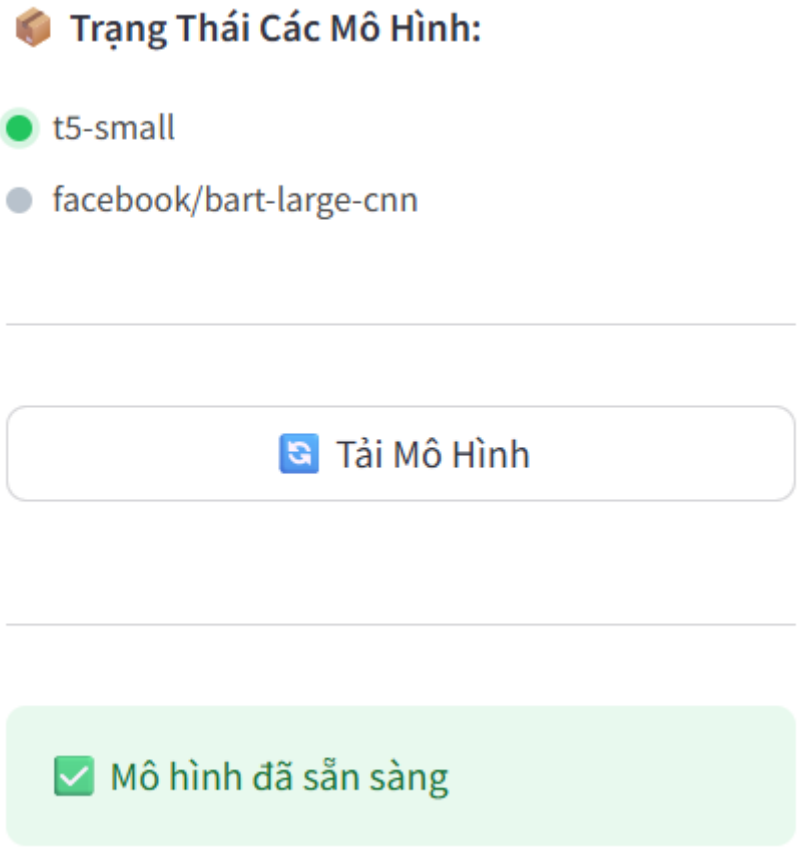
5.4.1. Kết quả kiểm thử chức năng giao diện



Hình 5.1 kết quả kiểm thử giao diện trang chủ

Qua quá trình chạy thử, ứng dụng khởi động thành công bằng Streamlit và hiển thị đầy đủ các thành phần giao diện như đã thiết kế trong app.py. Sidebar cho phép chuyển đổi ổn định giữa bốn trang chính là Trang Chủ, Công Cụ, Hướng Dẫn Sử Dụng và Về Ứng Dụng. Điều này cho thấy mã nguồn không chỉ xử lý phần backend mà đã được triển khai thành một ứng dụng web hoàn chỉnh ở mức cơ bản, đủ để phục vụ demo và đánh giá đồ án.

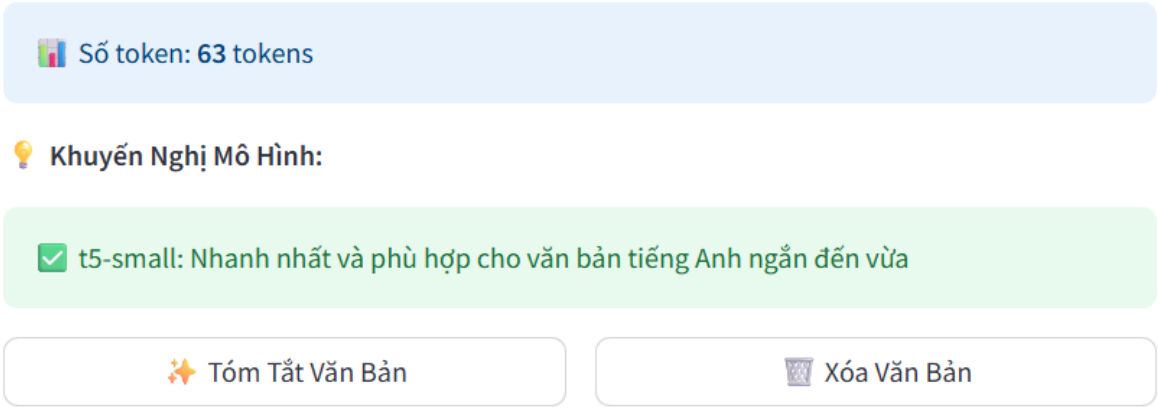
5.4.2. Kết quả tải mô hình



Hình 5.2 kết quả kiểm thử tải mô hình t5-small

Trong phiên bản hiện tại, hệ thống hỗ trợ hai mô hình là t5-small và facebook/bart-large-cnn. Quá trình thực nghiệm cho thấy t5-small có thời gian tải nhanh hơn và phù hợp hơn với môi trường tài nguyên hạn chế, trong khi facebook/bart-large-cnn thường cho bản tóm tắt bám nội dung nguồn tốt hơn nhưng yêu cầu nhiều RAM hơn và thời gian chờ dài hơn. Nhận xét này cũng phù hợp với phần mô tả trong README và giao diện ứng dụng.

5.4.3. Kết quả đếm token và gợi ý mô hình



Hình 5.3 kết quả kiểm thử chức năng đếm token

Một điểm nổi bật của hệ thống là khả năng đếm token trực tiếp sau khi người dùng nhập văn bản. Kết quả token được hiển thị ngay trên giao diện và hệ thống gợi ý mô hình theo ba ngưỡng chính: văn bản ngắn khoảng đến 1000 token, văn bản trung bình từ khoảng 1000 đến 3000 token và văn bản dài trên 3000 token. Cơ chế này hoạt động tốt trong thực nghiệm và giúp người dùng lựa chọn mô hình phù hợp hơn với độ dài đầu vào.

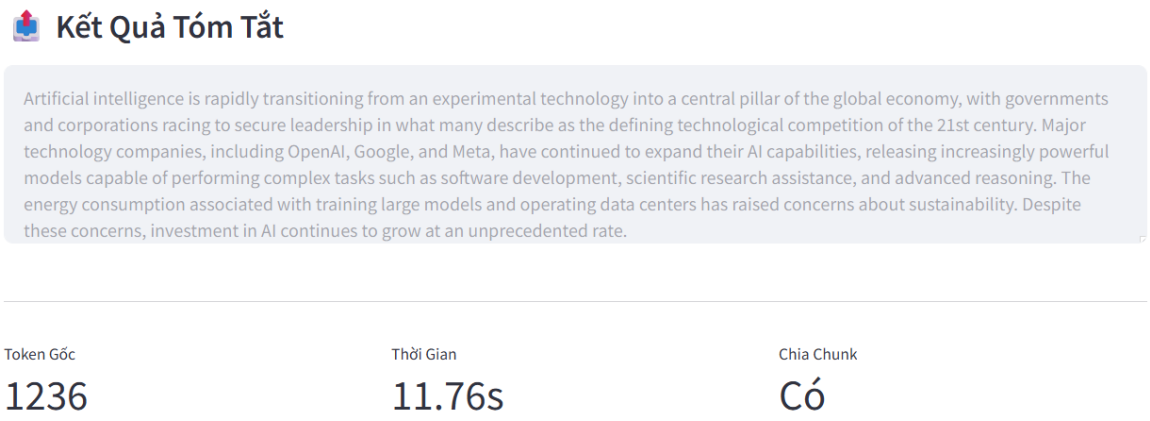
Chức năng này hoạt động tốt và giúp người dùng lựa chọn mô hình một cách thuận tiện hơn. Đây là một ưu điểm rõ rệt về mặt trải nghiệm người dùng, vì nó giúp giảm rủi ro chọn sai mô hình đối với văn bản dài hoặc tài nguyên máy yếu.

5.4.4. Kết quả với văn bản ngắn

Hình 5.4 kết quả kiểm thử việc tóm tắt văn bản ngắn

Khi đầu vào là văn bản ngắn, hệ thống thực hiện tóm tắt trực tiếp mà không cần chia nhỏ. Trong kết quả trả về, needs_chunking nhận giá trị False, danh sách chunks chỉ chứa văn bản gốc, chunk_summaries chỉ có một bản tóm tắt và final_summary chính là kết quả cuối cùng hiển thị trên giao diện. Trường hợp này cho thấy luồng xử lý đơn giản, tốc độ phản hồi nhanh và người dùng dễ quan sát kết quả.

5.4.5. Kết quả với văn bản dài



Hình 5.5 kết quả kiểm thử việc tóm tắt văn bản dài

Với đầu vào vượt ngưỡng max_tokens, hệ thống tiến hành chia văn bản thành nhiều chunk bằng cách cộng dồn số token theo từng câu để hạn chế việc cắt giữa chừng trong nội dung. Sau đó mỗi chunk được tóm tắt riêng, các bản tóm tắt trung gian được ghép lại và nếu cần thì tiếp tục rút gọn thêm một lần nữa để tạo final_summary. Trên giao diện, người dùng có thể quan sát bản tóm tắt cuối cùng, số token gốc, thời gian xử lý, trạng thái chunking, số lượng chunk cùng nội dung của từng phần. Kết quả này cho thấy thiết kế trong repo xử lý tương đối tốt bài toán đầu vào dài, vốn là hạn chế phổ biến của các mô hình Transformer.

5.4.6. Kết quả hiển thị thông tin hỗ trợ đánh giá

Ngoài bản tóm tắt cuối cùng, hệ thống còn hiển thị thời gian xử lý, số token của văn bản gốc, chi tiết từng chunk, trạng thái mô hình và trạng thái chia chunk. So với nhiều ứng dụng demo chỉ trả về một kết quả cuối, cách hiển thị này giúp quá trình xử lý minh bạch hơn và tạo thuận lợi cho việc giải thích sản phẩm trong buổi báo cáo.

5.5. Đánh giá hệ thống

5.5.1. Đánh giá định tính

Về mặt định tính, hệ thống cho thấy khả năng tạo ra bản tóm tắt ngắn gọn

hơn so với văn bản gốc, đồng thời vẫn giữ lại được những nội dung quan trọng ở mức tương đối tốt, đặc biệt khi sử dụng các mô hình BART. Đối với văn bản có cấu trúc rõ ràng và sử dụng tiếng Anh thông dụng, chất lượng đầu ra khá dễ đọc và phù hợp với mục đích nắm bắt ý chính.

Tuy nhiên, chất lượng tóm tắt vẫn phụ thuộc rõ rệt vào mô hình được chọn, độ dài văn bản đầu vào, mức độ rõ ràng của nội dung nguồn và cấu hình max_length hoặc min_length. Trong một số trường hợp, bản tóm tắt vẫn có thể mang tính rút gọn cơ học hoặc chưa thật sự mượt như cách diễn đạt của con người, đặc biệt khi văn bản quá dài hoặc có cấu trúc phức tạp.

5.5.2. Đánh giá định lượng

Để có cơ sở định lượng cụ thể hơn, đề tài sử dụng benchmark nội bộ được lưu tại benchmark_report.json thay cho việc chỉ quan sát cảm tính trên giao diện. Bộ benchmark này gồm 6 ca kiểm thử đại diện cho các nhóm dữ liệu tin tức, văn bản giải thích, hướng dẫn, FAQ, hội thoại và tự sự; mỗi mô hình được chấm theo reference score, fact score, support score, style score và accuracy tổng hợp. Cách đo này chưa thay thế cho ROUGE hoặc BLEU, nhưng phản ánh sát hơn trạng thái thực nghiệm thật của mã nguồn đang nộp.

Trong benchmark nội bộ, accuracy không được hiểu là độ chính xác phân loại, mà là điểm tổng hợp phản ánh mức độ bám tham chiếu, độ phủ thông tin, độ bám nguồn và độ phù hợp phong cách.

Công thức được sử dụng như sau: Accuracy = 0,45 × reference_score + 0,30 × fact_score + 0,15 × support_score + 0,10 × style_score.

Thành phần	Ý nghĩa	Trọng số
reference_score	Mức độ trùng khớp nội dung với bản tóm tắt tham chiếu	0,45
fact_score	Mức độ giữ lại các ý hoặc quan trọng của văn bản gốc	0,30
support_score	Mức độ các câu trong bản tóm tắt được câu nguồn hỗ trợ	0,15
style_score	Mức độ phong cách tóm tắt khớp với loại văn bản đầu vào	0,10

Bảng 5.1. Thành phần dùng để tính accuracy trong benchmark nội bộ.

Dựa trên số liệu trong benchmark_report.json, bảng dưới đây so sánh kết quả trung bình của hai mô hình hiện có trong hệ thống.

Mô hình	Accuracy	Reference	Fact	Support	Style
t5-small	62.99%	32.34%	81.67%	92.88%	100.00%
facebook/bart-large-cnn	61.64%	27.43%	81.11%	99.79%	100.00%

Bảng 5.2. So sánh độ chính xác và các thành phần đánh giá của hai mô hình.

Kết quả cho thấy t5-small đạt accuracy trung bình 62,99%, nhỉnh hơn nhẹ so với facebook/bart-large-cnn ở mức 61,64% trên bộ benchmark nội bộ. Tuy vậy, facebook/bart-large-cnn đạt support score 99,79%, cao hơn rõ rệt và cho thấy các câu sinh ra bám sát nguồn hơn. Điều này cho phép rút ra nhận xét thực tế rằng t5-small phù hợp khi cần cân bằng tốc độ và chất lượng chung, còn facebook/bart-large-cnn phù hợp khi ưu tiên độ bám nguồn và chấp nhận chi phí tài nguyên cao hơn.

5.5.3. Đánh giá trải nghiệm sử dụng

Về trải nghiệm người dùng, ứng dụng có nhiều điểm tích cực như giao diện trực quan, bố cục chia trang rõ ràng, thao tác tương đối dễ làm quen, thông báo trạng thái và cảnh báo lỗi được hiển thị cụ thể, đồng thời có thêm gợi ý lựa chọn mô hình theo độ dài văn bản và phần hiển thị chi tiết kết quả. Việc toàn bộ giao diện sử dụng tiếng Việt cũng giúp người dùng trong nước dễ tiếp cận hơn, dù phần xử lý tóm tắt hiện vẫn tối ưu cho văn bản tiếng Anh.

5.6. Ưu điểm của hệ thống

Qua quá trình thực nghiệm và phân tích repo hiện tại, có thể rút ra các ưu điểm chính của hệ thống như sau:

5.6.1. Có giao diện web hoàn chỉnh

Hệ thống không chỉ là một đoạn mã chạy trong môi trường lập trình mà đã được phát triển thành ứng dụng web hoàn chỉnh bằng Streamlit. Điều này giúp sản phẩm mang tính ứng dụng cao hơn và dễ trình bày trong đồ án.

5.6.2. Hỗ trợ nhiều mô hình

Việc tích hợp hai mô hình tạo ra sự linh hoạt trong lựa chọn giữa tốc độ và chất lượng. Trong thực nghiệm, t5-small phù hợp hơn khi cần tốc độ hoặc khi chạy trên môi trường tài nguyên thấp, còn facebook/bart-large-cnn thường bám sát nội dung nguồn hơn nhưng đánh đổi bằng thời gian tải và mức sử dụng bộ nhớ lớn hơn.

5.6.3. Có cơ chế chunking cho văn bản dài

Đây là ưu điểm quan trọng nhất. Thay vì bị giới hạn bởi kích thước đầu vào của mô hình, hệ thống có thể chia văn bản dài thành nhiều phần nhỏ để xử lý, sau đó tổng hợp lại. Điều này làm tăng đáng kể khả năng ứng dụng thực tế của chương trình.

5.6.4. Có khả năng hiển thị thông tin hỗ trợ đánh giá

Hệ thống không chỉ hiển thị kết quả cuối cùng mà còn cung cấp số token gốc, thời gian xử lý, số lượng chunk, nội dung của từng chunk và bản tóm tắt tương ứng. Nhờ đó quá trình xử lý được minh bạch hơn, người dùng và giảng viên có thể quan

sát rõ ứng dụng đã làm gì thay vì chỉ nhìn vào một đoạn tóm tắt cuối cùng.

5.6.5. Hỗ trợ CPU và GPU

Repo được xây dựng theo hướng có thể tự động dùng GPU nếu môi trường hỗ trợ. Điều này giúp tăng tốc độ xử lý và cho phép hệ thống hoạt động linh hoạt trên nhiều loại thiết bị khác nhau.

5.6.6. Tài liệu hướng dẫn khá đầy đủ

README.md cung cấp thông tin khá rõ về cách cài đặt, cách chạy, cấu trúc dự án và một số lưu ý khi triển khai. Đây là điểm thuận lợi trong việc hoàn thiện sản phẩm phần mềm vì người dùng mới có thể nhanh chóng nắm được quy trình thiết lập và thử nghiệm hệ thống.

5.7. Hạn chế của hệ thống

Bên cạnh các ưu điểm, hệ thống hiện tại vẫn còn một số hạn chế cần được nhìn nhận đúng với tình trạng repo thật.

5.7.1. Chưa hỗ trợ tốt cho tiếng Việt trong phần xử lý

Mặc dù giao diện được viết bằng tiếng Việt, phần tóm tắt hiện tại theo README và mô tả trong giao diện lại tối ưu cho **văn bản tiếng Anh**. Điều này tạo ra khoảng cách giữa đối tượng người dùng và dữ liệu đầu vào tối ưu của hệ thống.

5.7.2. Phụ thuộc vào mô hình pretrained

Hệ thống hiện sử dụng hoàn toàn các mô hình pretrained từ Hugging Face và chưa có bước huấn luyện hoặc tinh chỉnh riêng cho miền dữ liệu cụ thể. Vì vậy, chất lượng tóm tắt phụ thuộc lớn vào mô hình có sẵn và có thể chưa tối ưu cho mọi loại văn bản.

5.7.3. Tốn tài nguyên khi dùng mô hình lớn

Mô hình facebook/bart-large-cnn có dung lượng lớn và yêu cầu bộ nhớ cao. Điều này có thể gây khó khăn khi triển khai trên máy yếu hoặc dịch vụ cloud miễn phí.

5.7.4. Chưa có bộ đánh giá tự động chuẩn hóa

Hệ thống chưa tích hợp các chỉ số như ROUGE, BLEU hoặc các bộ dữ liệu chuẩn để đánh giá chất lượng tóm tắt. Vì vậy, việc đánh giá hiện mới dừng chủ yếu ở mức định tính và thực nghiệm trực tiếp.

5.7.5. Chưa hỗ trợ nhiều loại đầu vào

Hệ thống hiện chủ yếu nhận đầu vào dưới dạng văn bản nhập trực tiếp vào ô text area. Chưa có hỗ trợ đọc từ file .txt, .docx, .pdf hoặc URL tài liệu.

5.7.6. Chưa có cơ chế lưu lịch sử hoặc xuất kết quả

Bản tóm tắt chỉ hiển thị trên giao diện sau khi xử lý. Hệ thống chưa hỗ trợ lưu lịch sử phiên làm việc, tải xuống kết quả hoặc so sánh giữa nhiều lần tóm tắt.

5.8. So sánh với mục tiêu ban đầu

Đối chiếu với các mục tiêu đã đặt ra ở Chương 1, có thể thấy hệ thống đã đạt được nhiều kết quả quan trọng.

5.8.1. Các mục tiêu đã đạt được

- Đối chiếu với mục tiêu ban đầu, hệ thống đã đạt được những kết quả chính gồm xây dựng được một ứng dụng tóm tắt văn bản có thể chạy thực tế, triển khai giao diện web bằng Streamlit, tích hợp mô hình pretrained để sinh bản tóm tắt, hỗ trợ xử lý văn bản dài bằng chunking và hiển thị được các thông tin theo dõi như token, thời gian xử lý và chi tiết từng chunk. Đồng thời, mã nguồn cũng đã được tổ chức tương đối rõ ràng giữa phần giao diện, xử lý logic và cấu hình.

5.8.2. Các mục tiêu chưa hoàn thiện hoàn toàn

- Bên cạnh các mục tiêu đã hoàn thành, hệ thống vẫn còn một số phần chưa hoàn thiện hoàn toàn như chưa tích hợp các chỉ số đánh giá chuẩn vào giao diện, chưa tối ưu cho tiếng Việt trong khâu xử lý nội dung, chưa hỗ trợ nhập dữ liệu từ nhiều nguồn khác nhau và chưa tối ưu triệt để tài nguyên cho các mô hình lớn khi triển khai trên cloud. Những điểm này chủ yếu thuộc nhóm mở rộng và tối ưu sâu hơn so với phạm vi cốt lõi của đồ án.

5.9. Kết luận chương

Trong chương này, đề tài đã tiến hành thực nghiệm và đánh giá hệ thống tóm tắt văn bản dựa trên repo hiện tại. Kết quả cho thấy ứng dụng có thể hoạt động ổn định trên nền tảng Streamlit, hỗ trợ người dùng tải mô hình, nhập văn bản, cấu hình tham số và nhận bản tóm tắt đầu ra một cách trực quan.

Một trong những điểm nổi bật nhất của hệ thống là cơ chế chia nhỏ văn bản dài thành nhiều chunk để xử lý, qua đó khắc phục giới hạn đầu vào của các mô hình Transformer. Ngoài ra, hệ thống còn hỗ trợ nhiều mô hình khác nhau, hiển thị số token, thời gian xử lý và chi tiết từng chunk, giúp nâng cao tính minh bạch và tính thực tiễn của sản phẩm.

Bên cạnh những kết quả đạt được, hệ thống vẫn còn một số hạn chế như chưa hỗ trợ tốt cho tiếng Việt ở phần xử lý nội dung, chưa có bộ chỉ số đánh giá tự động và còn phụ thuộc vào tài nguyên phần cứng khi sử dụng các mô hình lớn. Tuy nhiên, xét trong phạm vi của một đồ án công nghệ thông tin, hệ thống hiện tại đã thể hiện được tính khả thi, tính ứng dụng và khả năng phát triển tiếp trong tương lai.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Những kết quả đạt được

Sau quá trình nghiên cứu, phân tích, thiết kế, xây dựng và thực nghiệm, đề tài "Xây dựng hệ thống tóm tắt văn bản tự động" đã đạt được một số kết quả quan trọng cả về mặt lý thuyết lẫn thực tiễn.

Trước hết, đề tài đã tìm hiểu được cơ sở lý thuyết liên quan đến xử lý ngôn ngữ tự nhiên và bài toán tóm tắt văn bản. Thông qua quá trình nghiên cứu, đề tài đã nắm được các hướng tiếp cận phổ biến trong tóm tắt văn bản, đặc biệt là việc ứng dụng các mô hình Transformer pretrained trong việc sinh bản tóm tắt.

Về mặt thiết kế và xây dựng hệ thống, đề tài đã xây dựng được một ứng dụng web hoàn chỉnh bằng ngôn ngữ Python, sử dụng Streamlit để phát triển giao diện và sử dụng các mô hình pretrained từ thư viện Transformers để thực hiện chức năng tóm tắt. Hệ thống hiện hỗ trợ hai mô hình gồm t5-small và facebook/bart-large-cnn, cho phép người dùng lựa chọn mô hình phù hợp tùy theo nhu cầu sử dụng và độ dài văn bản đầu vào.

Một kết quả đáng chú ý khác là hệ thống đã triển khai thành công cơ chế đếm token và chia nhỏ văn bản thành các chunk khi nội dung đầu vào vượt quá giới hạn xử lý trực tiếp của mô hình. Cơ chế này giúp hệ thống xử lý được các văn bản dài hơn, đồng thời làm tăng tính ứng dụng thực tế của sản phẩm.

Bên cạnh đó, giao diện của hệ thống được xây dựng tương đối trực quan và dễ sử dụng. Người dùng có thể lựa chọn mô hình, cấu hình tham số, nhập văn bản, thực hiện tóm tắt và xem kết quả trực tiếp trên trình duyệt. Hệ thống cũng hỗ trợ hiển thị một số thông tin hữu ích như số token của văn bản gốc, thời gian xử lý, trạng thái chia chunk và chi tiết bản tóm tắt của từng chunk.

Nhìn chung, đề tài đã hoàn thành được mục tiêu chính là xây dựng một hệ thống tóm tắt văn bản có khả năng hoạt động thực tế, có giao diện tương tác rõ ràng và có nền tảng kỹ thuật đủ tốt để tiếp tục mở rộng trong tương lai.

6.2. Hạn chế còn tồn tại

Mặc dù đã đạt được những kết quả nhất định, hệ thống hiện tại vẫn còn một số hạn chế cần được nhìn nhận khách quan.

Trước hết, theo cấu trúc và tài liệu của repo hiện tại, hệ thống được xây dựng với giao diện tiếng Việt nhưng chức năng tóm tắt vẫn chủ yếu phù hợp với văn bản tiếng Anh. Điều này làm giảm phần nào tính ứng dụng đối với người dùng có nhu cầu xử lý trực tiếp văn bản tiếng Việt.

Thứ hai, hệ thống hiện mới sử dụng các mô hình pretrained có sẵn từ Hugging Face mà chưa thực hiện huấn luyện bổ sung hoặc tinh chỉnh mô hình trên tập dữ liệu riêng. Vì vậy, chất lượng tóm tắt phụ thuộc khá nhiều vào khả năng tổng quát của mô hình gốc và có thể chưa thật sự tối ưu với các loại văn bản chuyên ngành hoặc cấu trúc phức tạp.

Thứ ba, hệ thống chưa tích hợp các phương pháp đánh giá tự động bằng chỉ số chuẩn như ROUGE, BLEU hoặc BERTScore. Do đó, việc đánh giá chất lượng bản tóm tắt hiện tại chủ yếu dựa vào quan sát thực nghiệm, độ dài đầu ra, thời gian xử lý và nhận xét định tính.

Ngoài ra, các mô hình lớn như facebook/bart-large-cnn yêu cầu dung lượng bộ nhớ và thời gian tải khá lớn. Điều này có thể gây khó khăn khi triển khai trên các môi trường có tài nguyên hạn chế như máy cá nhân cấu hình thấp hoặc nền tảng cloud miễn phí.

Bên cạnh đó, hệ thống hiện chỉ hỗ trợ đầu vào ở dạng văn bản nhập trực tiếp trên giao diện. Chưa có chức năng nhập từ file .txt, .docx, .pdf hoặc lấy dữ liệu từ URL. Đồng thời, hệ thống cũng chưa hỗ trợ lưu kết quả, tải xuống bản tóm tắt hoặc quản lý lịch sử sử dụng.

Những hạn chế này là phù hợp với phạm vi của một đồ án học phần, tuy nhiên cũng cho thấy còn nhiều hướng để tiếp tục cải tiến và hoàn thiện hệ thống.

6.3. Hướng phát triển

Từ những hạn chế đã nêu, đề tài có thể tiếp tục được mở rộng theo nhiều hướng nhằm nâng cao chất lượng và khả năng ứng dụng thực tế của hệ thống.

Hướng phát triển đầu tiên và quan trọng nhất là nâng cao khả năng hỗ trợ tiếng Việt. Hệ thống có thể được cải tiến bằng cách tích hợp các mô hình phù hợp hơn với tiếng Việt hoặc tiến hành fine-tune trên các bộ dữ liệu tóm tắt tiếng Việt, từ đó nâng cao chất lượng đầu ra cho người dùng trong nước.

Tiếp theo, hệ thống nên được bổ sung các chỉ số đánh giá tự động như ROUGE, BLEU hoặc các phương pháp đánh giá theo ngữ nghĩa hiện đại hơn. Điều này sẽ giúp việc đánh giá chất lượng bản tóm tắt trở nên khách quan, khoa học và có cơ sở so sánh rõ ràng hơn giữa các mô hình.

Một hướng phát triển khác là mở rộng nguồn dữ liệu đầu vào. Thay vì chỉ cho phép người dùng nhập văn bản trực tiếp, hệ thống có thể hỗ trợ đọc từ các định dạng tài liệu phổ biến như .txt, .docx, .pdf, hoặc lấy nội dung từ URL bài báo, trang web hay các nguồn dữ liệu trực tuyến khác.

Ngoài các hướng phát triển về mô hình và đánh giá, hệ thống còn có thể được

mở rộng theo hướng hỗ trợ người dùng tốt hơn, chẳng hạn như lưu lịch sử tóm tắt, xuất kết quả ra file, cho phép tải bản tóm tắt xuống máy, so sánh đầu ra giữa nhiều mô hình và bổ sung thêm các tùy chọn điều chỉnh độ dài hoặc phong cách tóm tắt ngay trên giao diện.

Về mặt kỹ thuật, hệ thống có thể tiếp tục được tối ưu bằng cách giảm thời gian tải mô hình, giảm mức sử dụng bộ nhớ, tăng hiệu suất xử lý và cải thiện cơ chế chunking để bảo toàn ngữ nghĩa tốt hơn khi xử lý các văn bản rất dài.

Trong tương lai, nếu có điều kiện phát triển ở quy mô lớn hơn, hệ thống có thể được tách thành kiến trúc frontend – backend riêng biệt, xây dựng API phục vụ nhiều client khác nhau, triển khai trên hạ tầng cloud chuyên dụng và tích hợp thêm các cơ chế quản lý người dùng hoặc lưu trữ dữ liệu.

6.4. Kết luận

Tóm lại, đề tài "Xây dựng hệ thống tóm tắt văn bản tự động" đã hoàn thành được mục tiêu chính là xây dựng một ứng dụng web hỗ trợ tóm tắt văn bản bằng cách kết hợp giao diện Streamlit với các mô hình AI pretrained thuộc thư viện Transformers. Hệ thống đã thể hiện được tính khả thi cả về mặt kỹ thuật lẫn ứng dụng thực tế, đồng thời cho thấy khả năng vận dụng các kiến thức về lập trình, xử lý ngôn ngữ tự nhiên và phát triển phần mềm vào một sản phẩm cụ thể.

Thông qua việc xây dựng hệ thống, đề tài đã không chỉ dừng lại ở mức nghiên cứu lý thuyết mà còn triển khai được một ứng dụng có khả năng hoạt động thực tế, có giao diện trực quan, có hỗ trợ xử lý văn bản dài thông qua cơ chế chia chunk và có thể chạy trên nhiều môi trường khác nhau. Đây là một kết quả có ý nghĩa trong phạm vi của một đồ án công nghệ thông tin.

Mặc dù hệ thống vẫn còn những hạn chế nhất định, đặc biệt ở khả năng hỗ trợ tiếng Việt, cơ chế đánh giá tự động và tối ưu tài nguyên, nhưng những kết quả đạt được cho thấy đề tài có nền tảng tốt để tiếp tục phát triển. Nếu được đầu tư thêm về dữ liệu, mô hình và hạ tầng triển khai, hệ thống hoàn toàn có thể được mở rộng thành một ứng dụng hoàn thiện hơn, phục vụ tốt hơn cho nhu cầu học tập, nghiên cứu và khai thác thông tin từ văn bản.

Qua toàn bộ quá trình thực hiện đề tài, có thể khẳng định rằng hệ thống tóm tắt văn bản được xây dựng là một sản phẩm có giá trị thực tiễn, đồng thời là cơ sở quan trọng để tiếp tục nghiên cứu và phát triển các ứng dụng xử lý ngôn ngữ tự nhiên trong tương lai.

TÀI LIỆU THAM KHẢO

[1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” Proceedings of NAACL-HLT, 2019.

[2] Hugging Face, “Transformers Documentation,” [Online]. Available: Hugging Face official documentation.

[3] PyTorch, “PyTorch Documentation,” [Online]. Available: PyTorch official documentation.

[4] Streamlit, “Streamlit Documentation,” [Online]. Available: Streamlit official documentation.